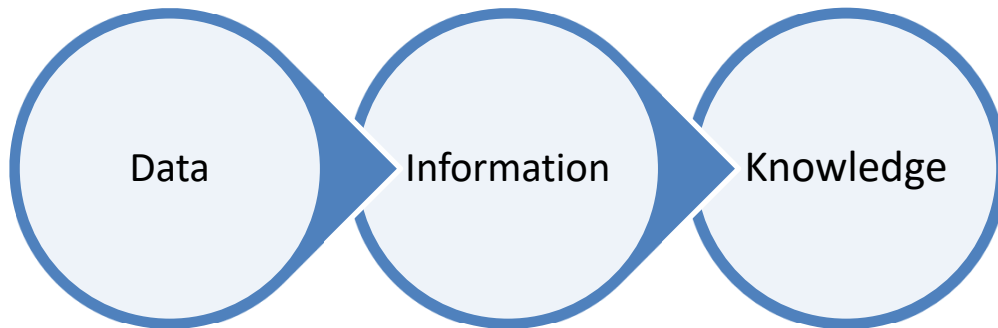


Introduction to Data Science: Data and Challenges in Managing Data



Introduction to Data Science: Data Transformation



Introduction to Data Science: Data Sources



Introduction to Data Science: Big Data

“At the beginning of **2020**, the digital universe was estimated to consist of 44 zettabytes of **data**. By 2025, approximately 463 exabytes would be **created every** 24 hours worldwide.”

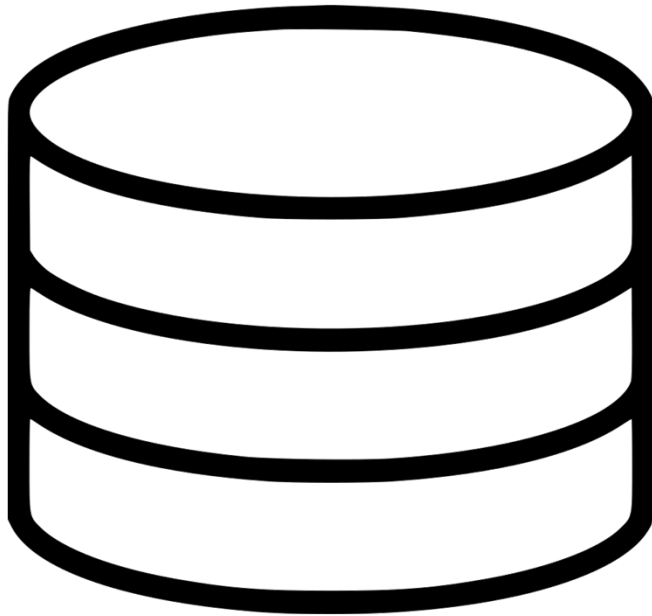
Source: seedscientific.com

Introduction to Data Science: Challenges in Managing Data



- Amount of data increases exponentially
- Data are scattered, unstructured, and from multiple sources
- Data security, quality and integrity issues

Introduction to Data Science: Challenges in Managing Data

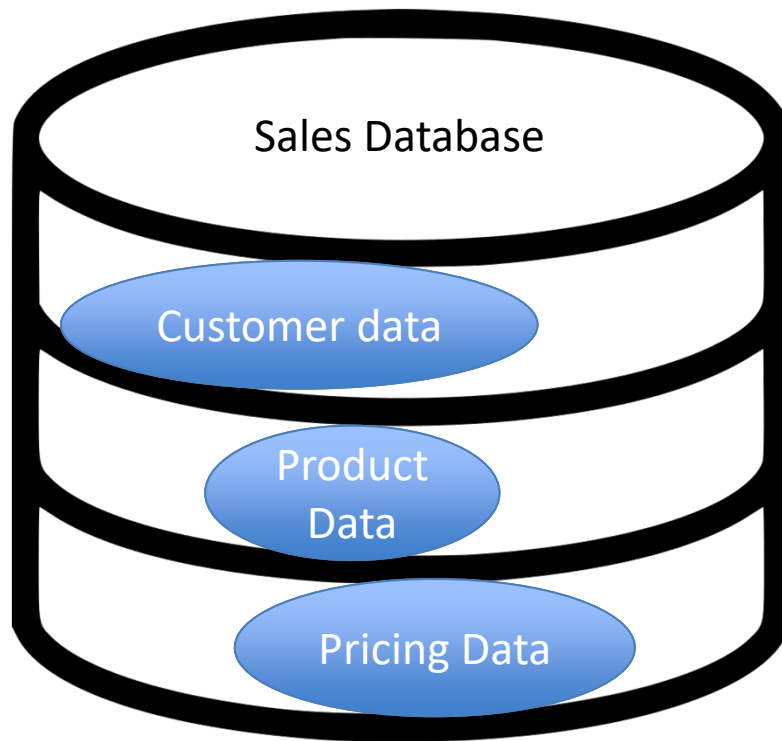


Concept of Database to organize and
manage Data

Introduction to Data Science: Database Concept

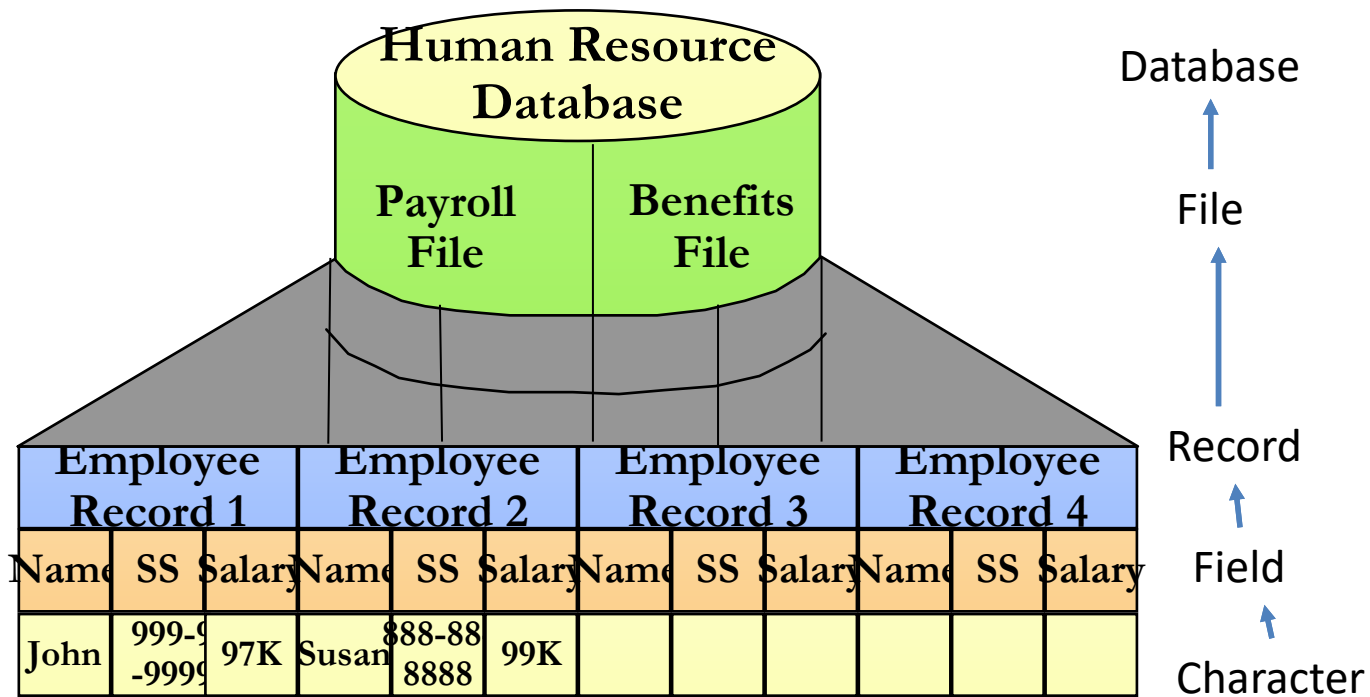


Introduction to Data Science: Database Concept

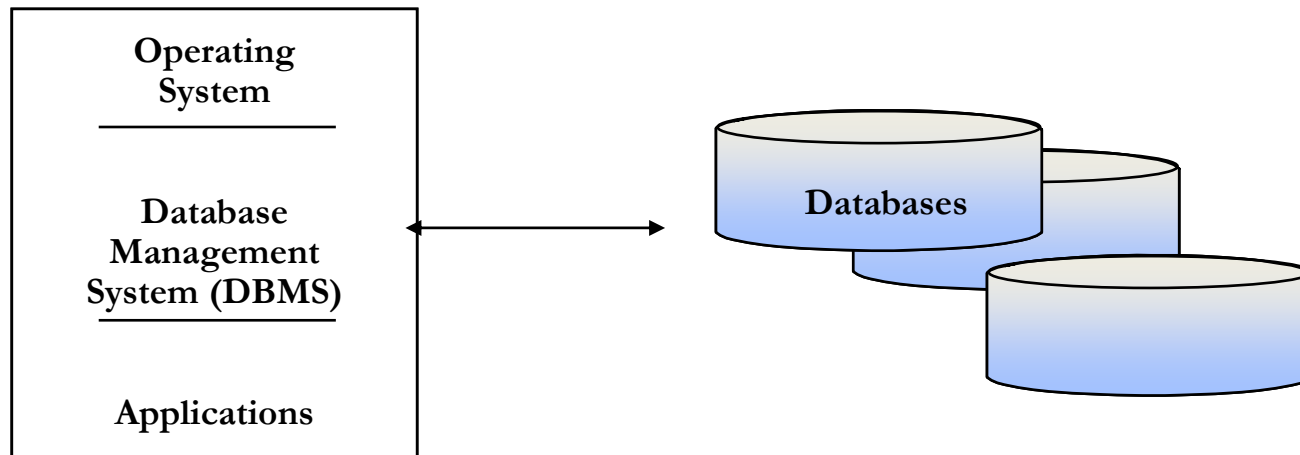


A database is an **integrated collection** of logically related records or objects.

Introduction to Data Science: Relational Database



Introduction to Data Science: Database Concept



Introduction to Data Science: Database Management System (DBMS)

Functions of a DBMS

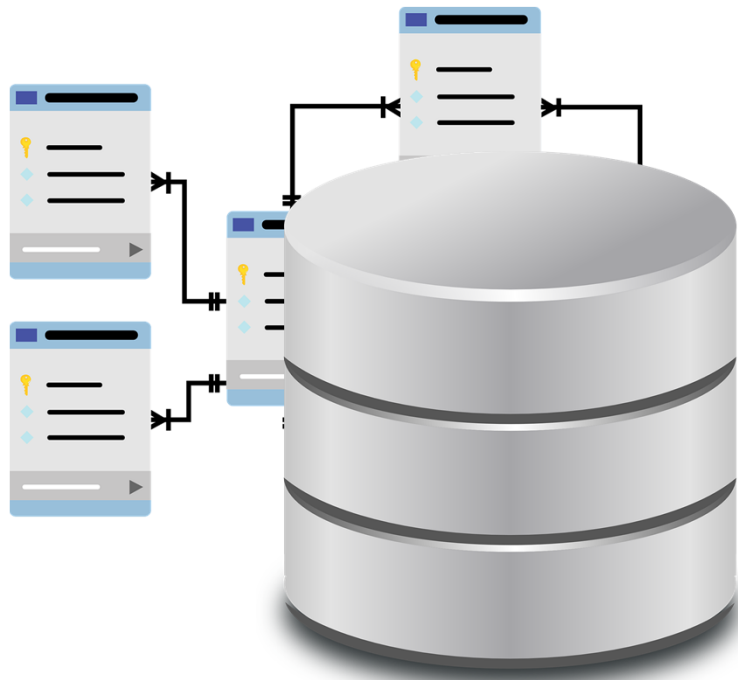
- Create Databases
- Database Queries
- Database
Maintenance
- Provides Interface to
Applications

Introduction to Data Science: Database Management System (DBMS)

Benefits

- Databases Integration
- Reduces Redundancy
- Enables Data Sharing
- Manages Integrity

Introduction to Data Science: Structured Query Language

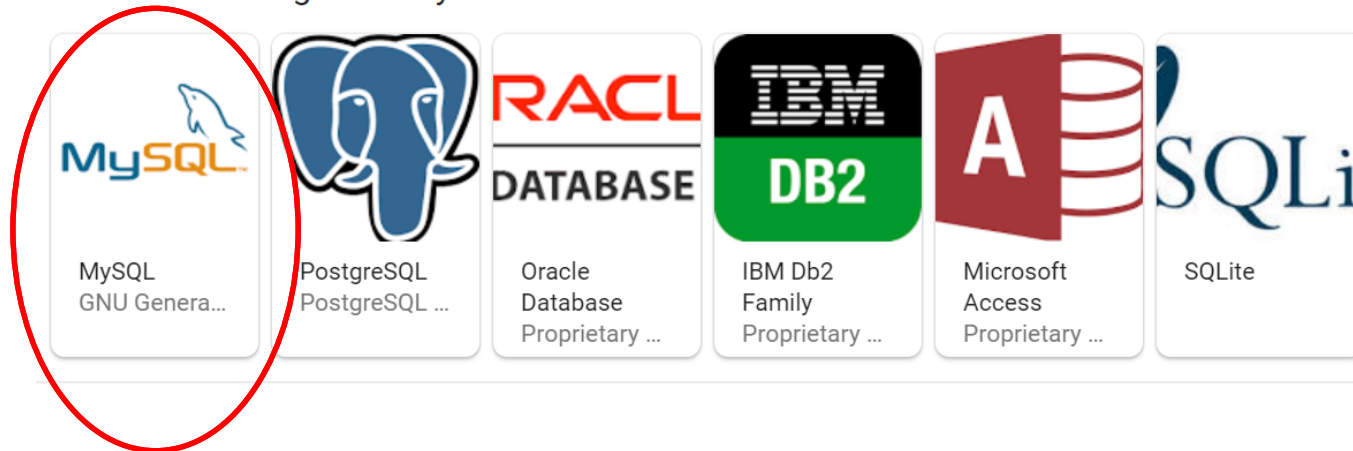


Structured Query Language (SQL)

- Developed by IBM (SEQUEL)
- Standard database programming language for interface with relational databases.
 - create, maintain, retrieve

Introduction to Data Science: SQL Database Management Systems

Database management system Software



What do they have in common?

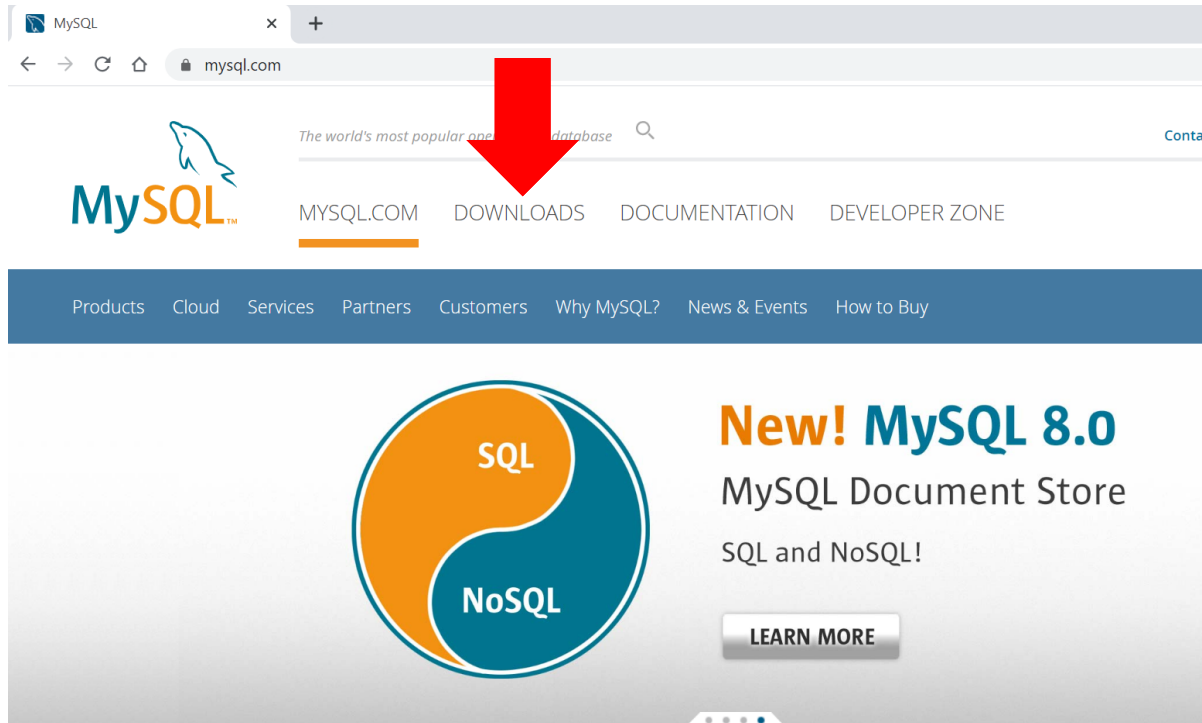
- Structured Query Language (SQL) as the database programming language

Introduction to Data Science: Why MySQL?



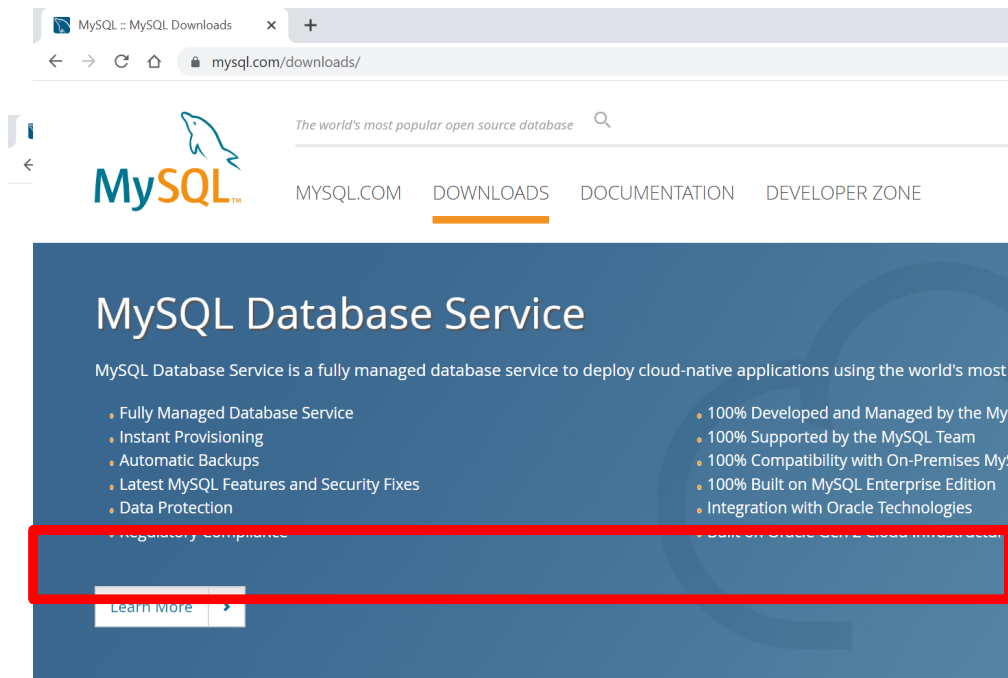
Rank			DBMS	Database Model
May 2020	Apr 2020	May 2019		
1.	1.	1.	Oracle +	Relational, Multi-model ⓘ
2.	2.	2.	MySQL +	Relational, Multi-model ⓘ
3.	3.	3.	Microsoft SQL Server +	Relational, Multi-model ⓘ
4.	4.	4.	PostgreSQL +	Relational, Multi-model ⓘ
5.	5.	5.	MongoDB +	Document, Multi-model ⓘ
6.	6.	6.	IBM Db2 +	Relational, Multi-model ⓘ
7.	7.	7.	Elasticsearch +	Search engine, Multi-model ⓘ
8.	8.	8.	Redis +	Key-value, Multi-model ⓘ
9.	9.	↑ 11.	SQLite +	Relational
10.	10.	↓ 9.	Microsoft Access	Relational

Introduction to Data Science: MySQL installation



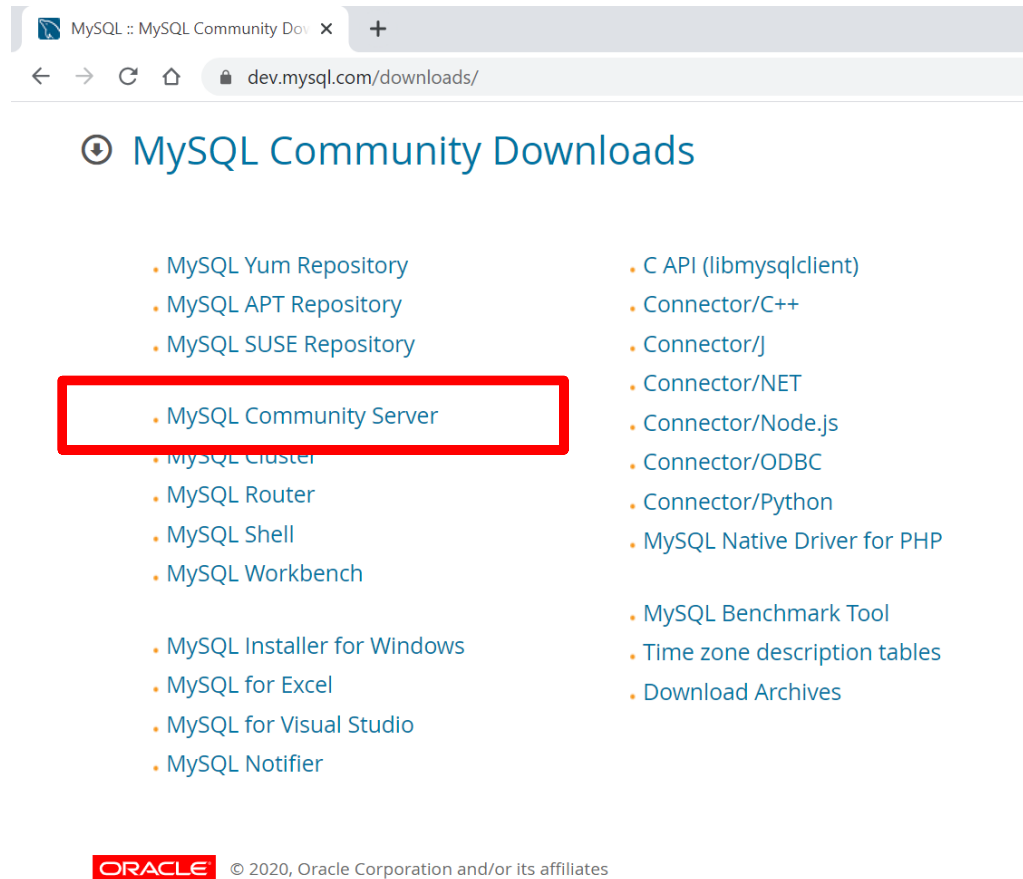
URL: mysql.com

Introduction to Data Science: MySQL installation



[MySQL Community \(GPL\) Downloads »](#)

Introduction to Data Science: MySQL installation



The screenshot shows a web browser window with the address bar displaying 'dev.mysql.com/downloads/'. The page title is 'MySQL :: MySQL Community Downloads'. The main heading is 'MySQL Community Downloads'. Below this, there are two columns of links. The first column includes 'MySQL Yum Repository', 'MySQL APT Repository', 'MySQL SUSE Repository', 'MySQL Community Server' (highlighted with a red box), 'MySQL Cluster', 'MySQL Router', 'MySQL Shell', 'MySQL Workbench', 'MySQL Installer for Windows', 'MySQL for Excel', 'MySQL for Visual Studio', and 'MySQL Notifier'. The second column includes 'C API (libmysqlclient)', 'Connector/C++', 'Connector/J', 'Connector/NET', 'Connector/Node.js', 'Connector/ODBC', 'Connector/Python', 'MySQL Native Driver for PHP', 'MySQL Benchmark Tool', 'Time zone description tables', and 'Download Archives'. At the bottom, there is an Oracle logo and the copyright notice '© 2020, Oracle Corporation and/or its affiliates'.

MySQL :: MySQL Community Downloads

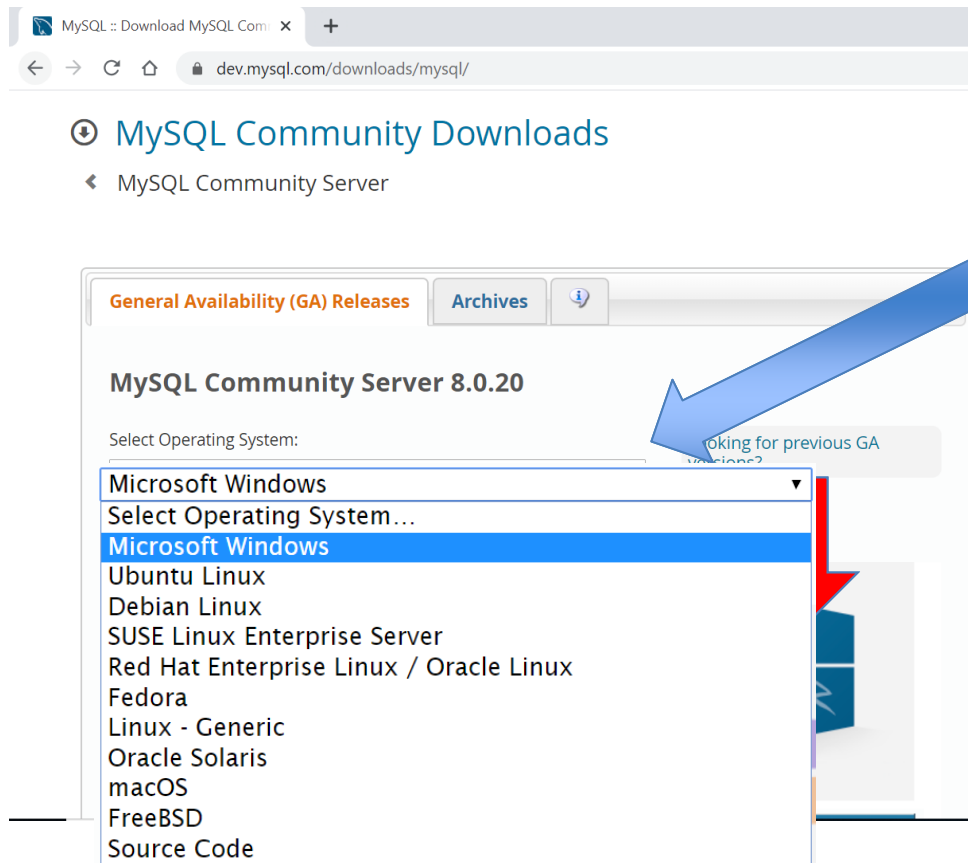
dev.mysql.com/downloads/

MySQL Community Downloads

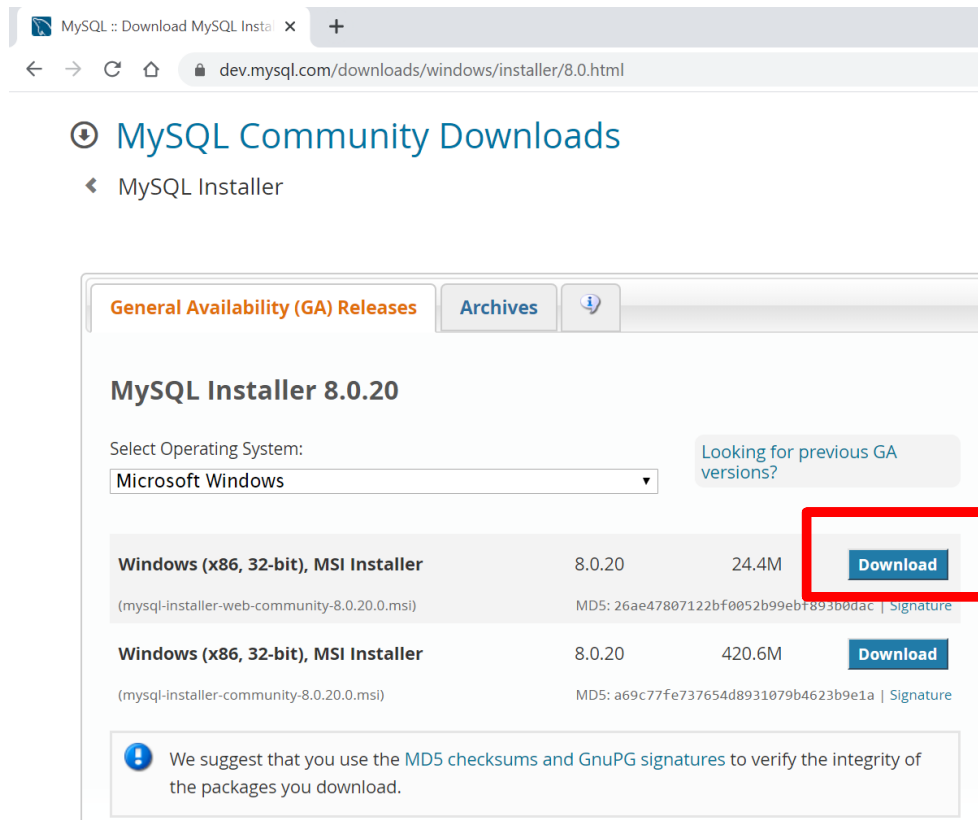
- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository
- MySQL Community Server
- MySQL Cluster
- MySQL Router
- MySQL Shell
- MySQL Workbench
- MySQL Installer for Windows
- MySQL for Excel
- MySQL for Visual Studio
- MySQL Notifier
- C API (libmysqlclient)
- Connector/C++
- Connector/J
- Connector/NET
- Connector/Node.js
- Connector/ODBC
- Connector/Python
- MySQL Native Driver for PHP
- MySQL Benchmark Tool
- Time zone description tables
- Download Archives

ORACLE © 2020, Oracle Corporation and/or its affiliates

Introduction to Data Science: MySQL installation



Introduction to Data Science: MySQL installation

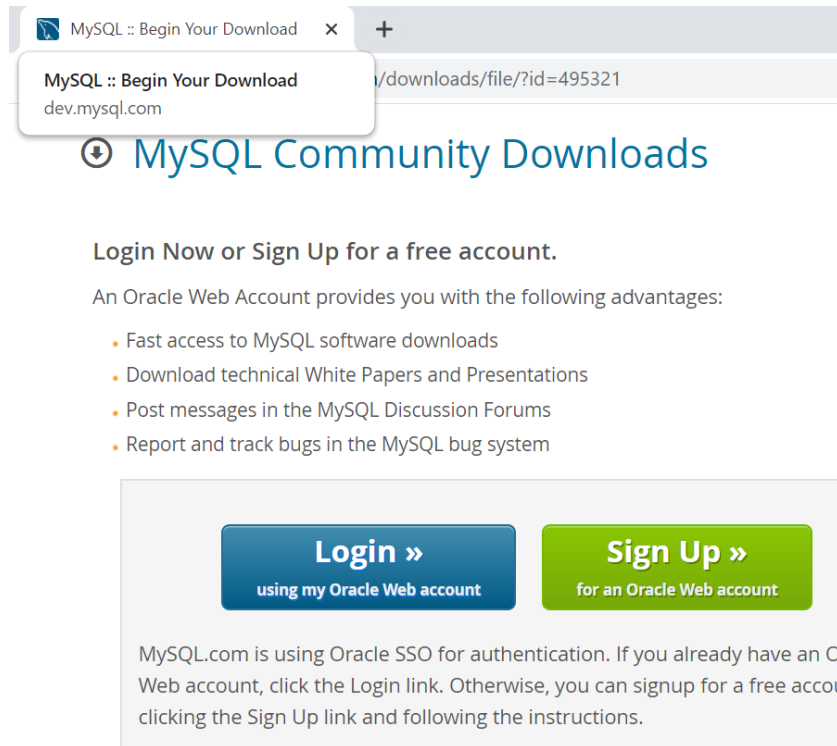


The screenshot shows a web browser window with the address bar displaying `dev.mysql.com/downloads/windows/installer/8.0.html`. The page title is "MySQL Community Downloads" with a sub-breadcrumb "MySQL Installer". The main content area has tabs for "General Availability (GA) Releases", "Archives", and a help icon. Under "General Availability (GA) Releases", the heading "MySQL Installer 8.0.20" is followed by a "Select Operating System:" dropdown menu set to "Microsoft Windows" and a link "Looking for previous GA versions?". Below this is a table of download links. The first row, "Windows (x86, 32-bit), MSI Installer", is highlighted with a red box around its "Download" button. The second row, "Windows (x86, 32-bit), MSI Installer", also has a "Download" button. At the bottom, a blue information icon is followed by a message: "We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download."

Operating System	Version	Size	Action
Windows (x86, 32-bit), MSI Installer	8.0.20	24.4M	Download
Windows (x86, 32-bit), MSI Installer	8.0.20	420.6M	Download

We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

Introduction to Data Science: MySQL installation



The screenshot shows a web browser window with a single tab titled "MySQL :: Begin Your Download". The address bar shows a URL ending in "/downloads/file/?id=495321". A tooltip is visible over the tab, showing the page title and the URL "dev.mysql.com". Below the browser window, the page header reads "MySQL Community Downloads". The main content area prompts the user to "Login Now or Sign Up for a free account." and lists the advantages of an Oracle Web Account: fast access to MySQL software downloads, downloading technical white papers and presentations, posting messages in the MySQL Discussion Forums, and reporting and tracking bugs in the MySQL bug system. Two buttons are provided: a blue "Login »" button for users with an Oracle Web account and a green "Sign Up »" button for new users. A paragraph below the buttons explains that MySQL.com uses Oracle SSO for authentication and provides instructions for both existing and new users.

MySQL :: Begin Your Download x +

MySQL :: Begin Your Download
dev.mysql.com

/downloads/file/?id=495321

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »
using my Oracle Web account

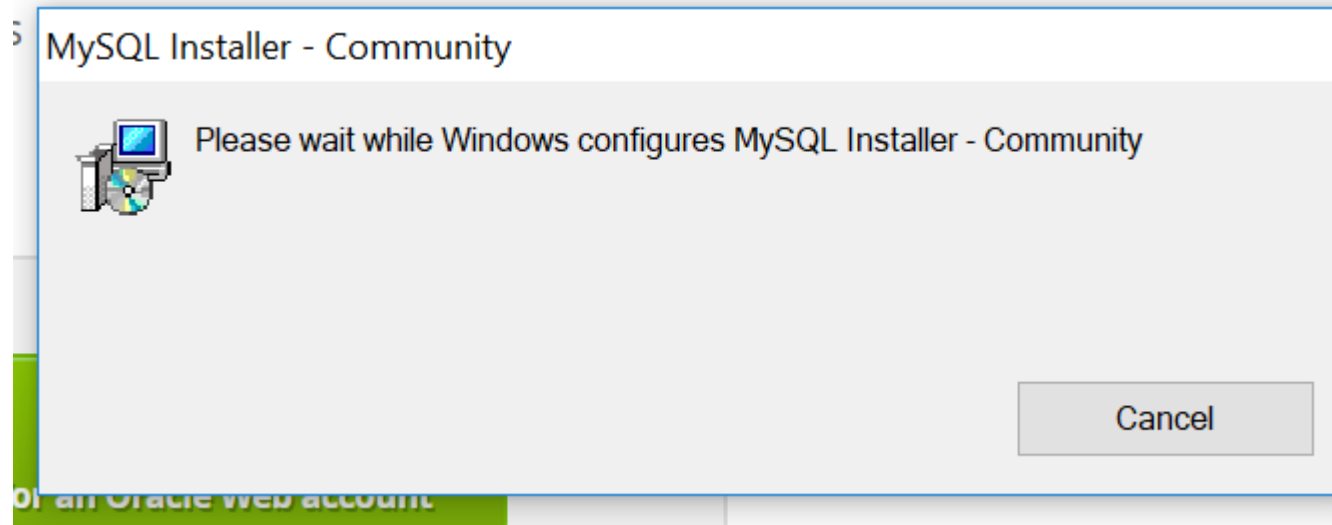
Sign Up »
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can sign up for a free account by clicking the Sign Up link and following the instructions.

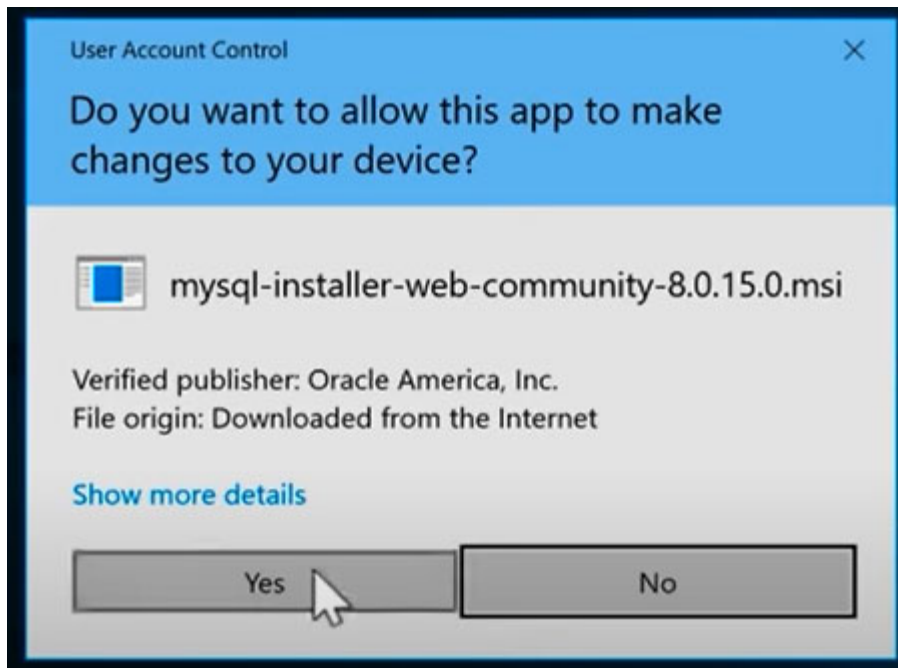


mysql-installer-we....msi

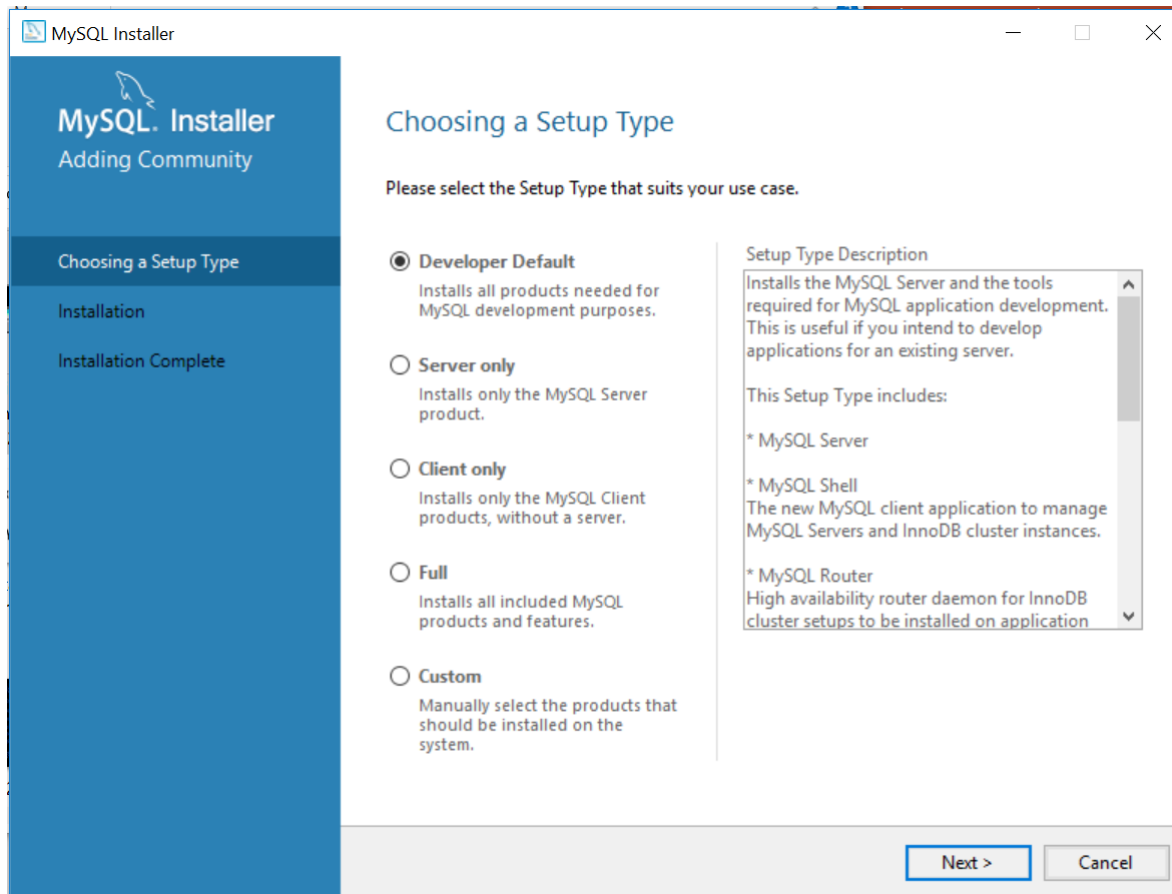
Introduction to Data Science: MySQL installation



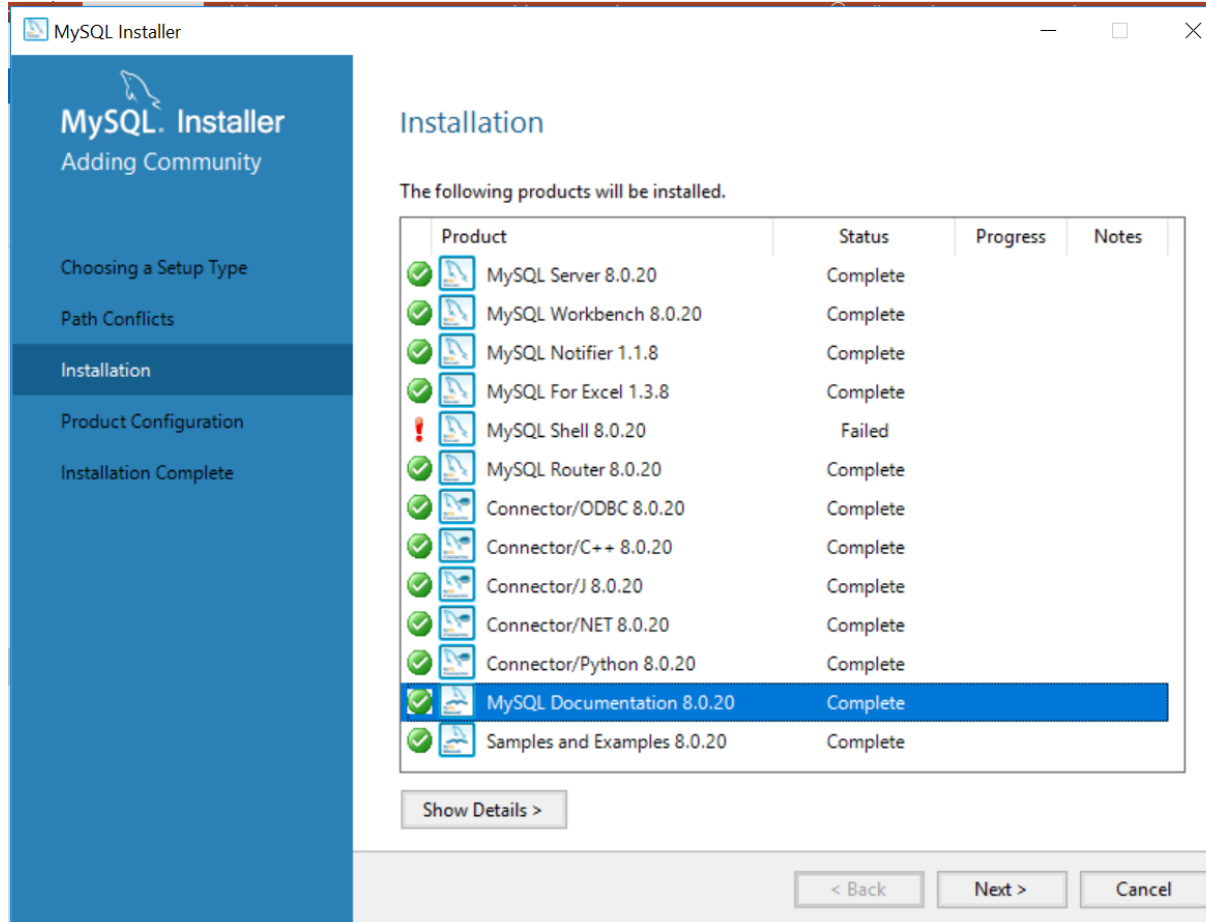
Introduction to Data Science: MySQL installation



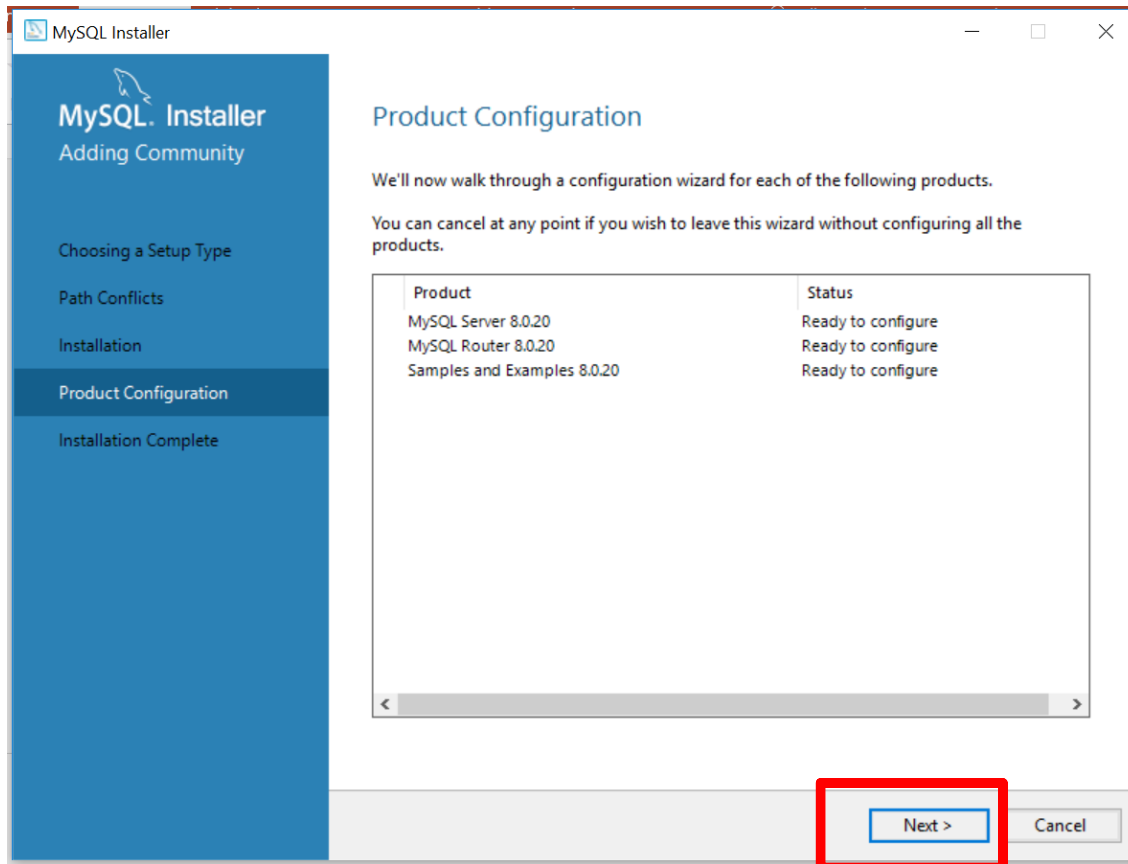
Introduction to Data Science: MySQL installation



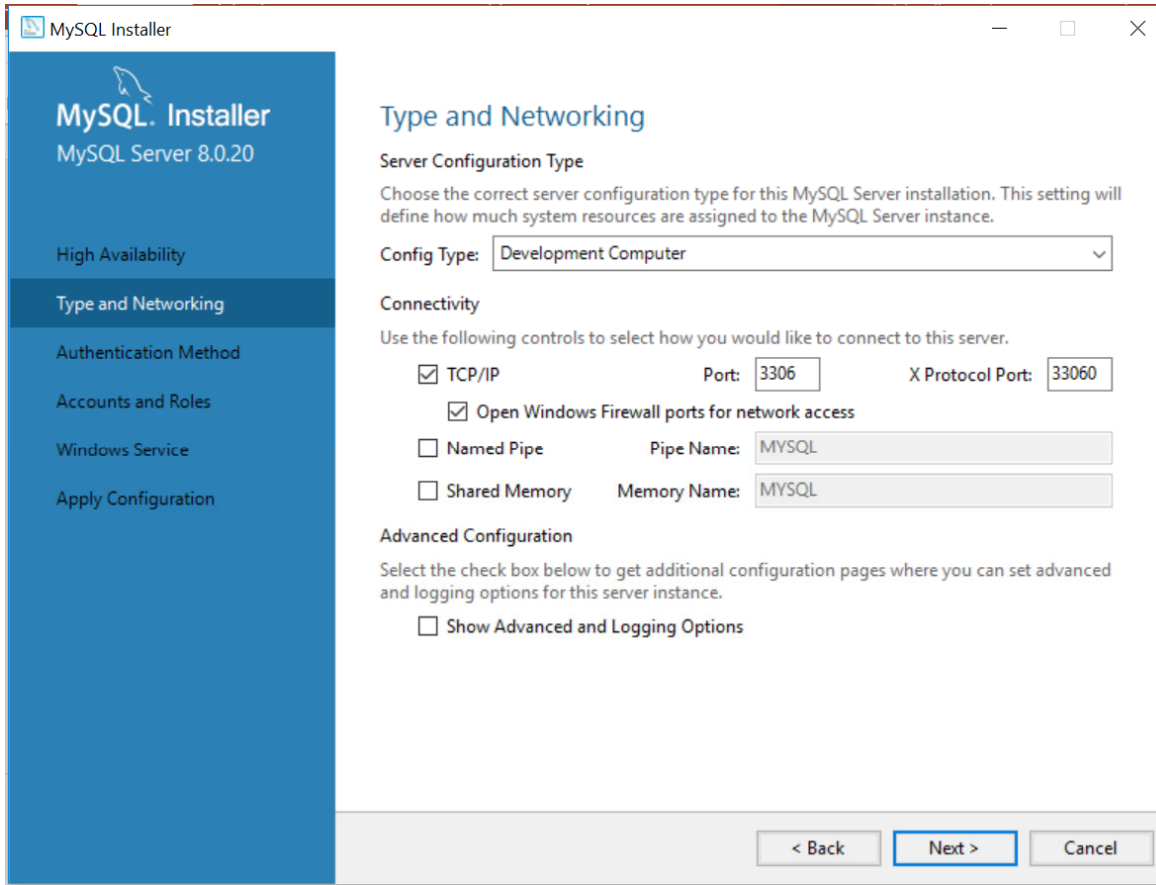
Introduction to Data Science: MySQL installation



Introduction to Data Science: MySQL installation



Introduction to Data Science: MySQL installation

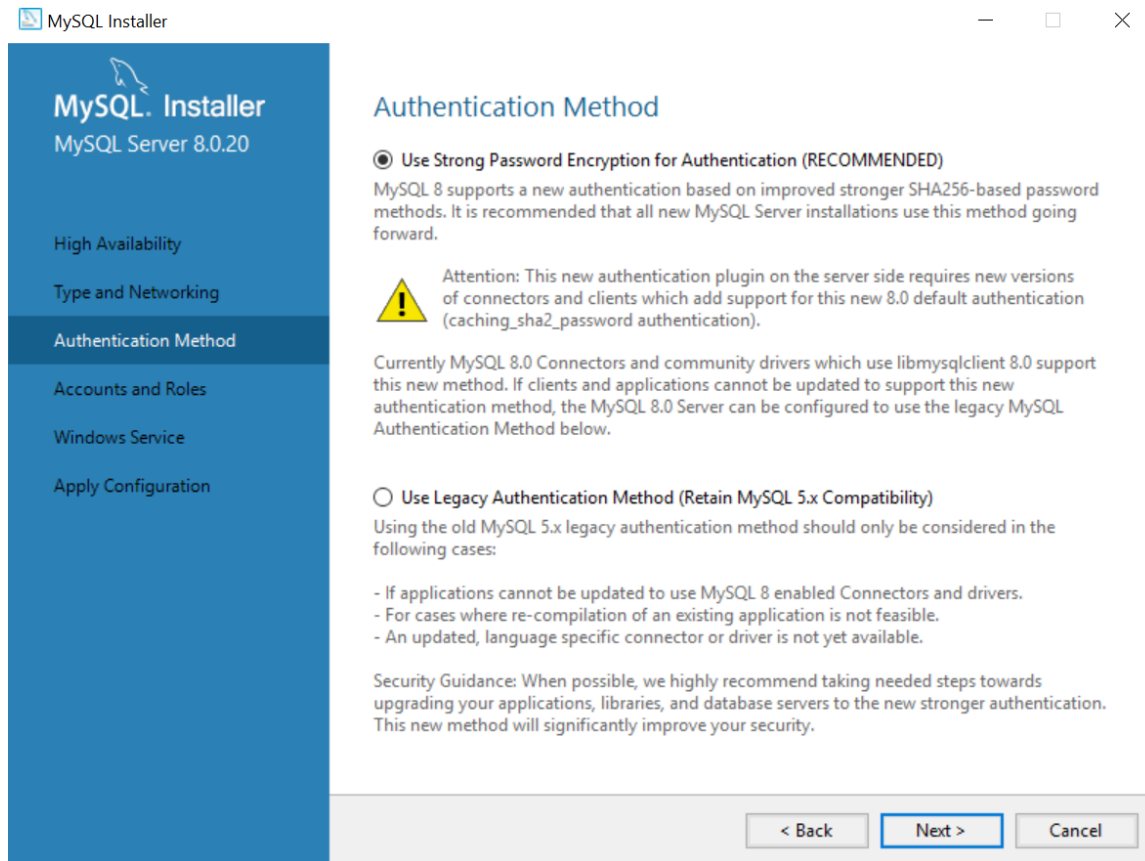


The image shows the MySQL Installer window for MySQL Server 8.0.20. The left sidebar contains the following navigation items: MySQL Installer, MySQL Server 8.0.20, High Availability, Type and Networking (selected), Authentication Method, Accounts and Roles, Windows Service, and Apply Configuration. The main content area is titled 'Type and Networking' and includes the following sections:

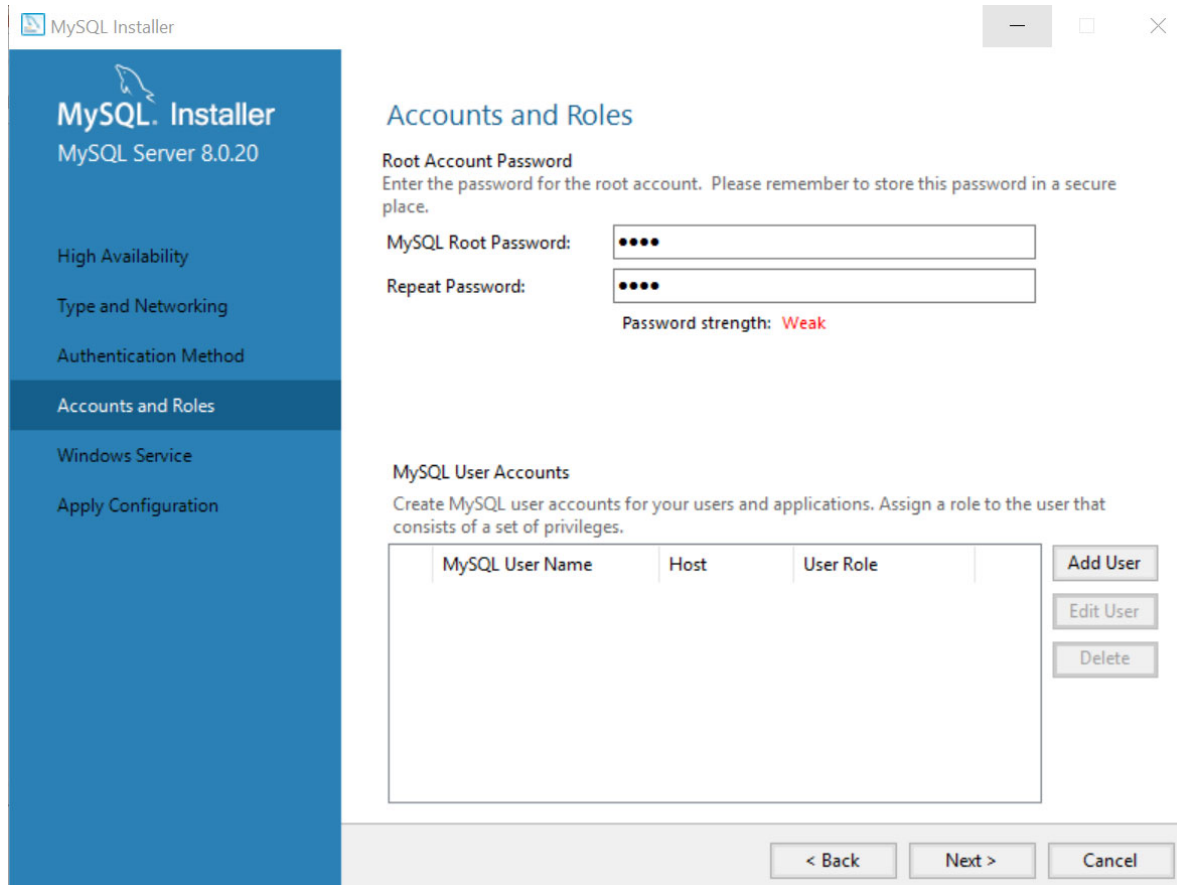
- Server Configuration Type**
Choose the correct server configuration type for this MySQL Server installation. This setting will define how much system resources are assigned to the MySQL Server instance.
Config Type: Development Computer
- Connectivity**
Use the following controls to select how you would like to connect to this server.
 - ☒ TCP/IP Port: 3306 X Protocol Port: 33060
 - ☒ Open Windows Firewall ports for network access
 - ☐ Named Pipe Pipe Name: MYSQL
 - ☐ Shared Memory Memory Name: MYSQL
- Advanced Configuration**
Select the check box below to get additional configuration pages where you can set advanced and logging options for this server instance.
 - ☐ Show Advanced and Logging Options

At the bottom of the window are three buttons: < Back, Next > (highlighted), and Cancel.

Introduction to Data Science: MySQL installation



Introduction to Data Science: MySQL installation



The image shows the MySQL Installer window for MySQL Server 8.0.20. The left sidebar contains the following options: High Availability, Type and Networking, Authentication Method, Accounts and Roles (selected), Windows Service, and Apply Configuration. The main area is titled 'Accounts and Roles' and contains two sections: 'Root Account Password' and 'MySQL User Accounts'.

Root Account Password
Enter the password for the root account. Please remember to store this password in a secure place.

MySQL Root Password:

Repeat Password:

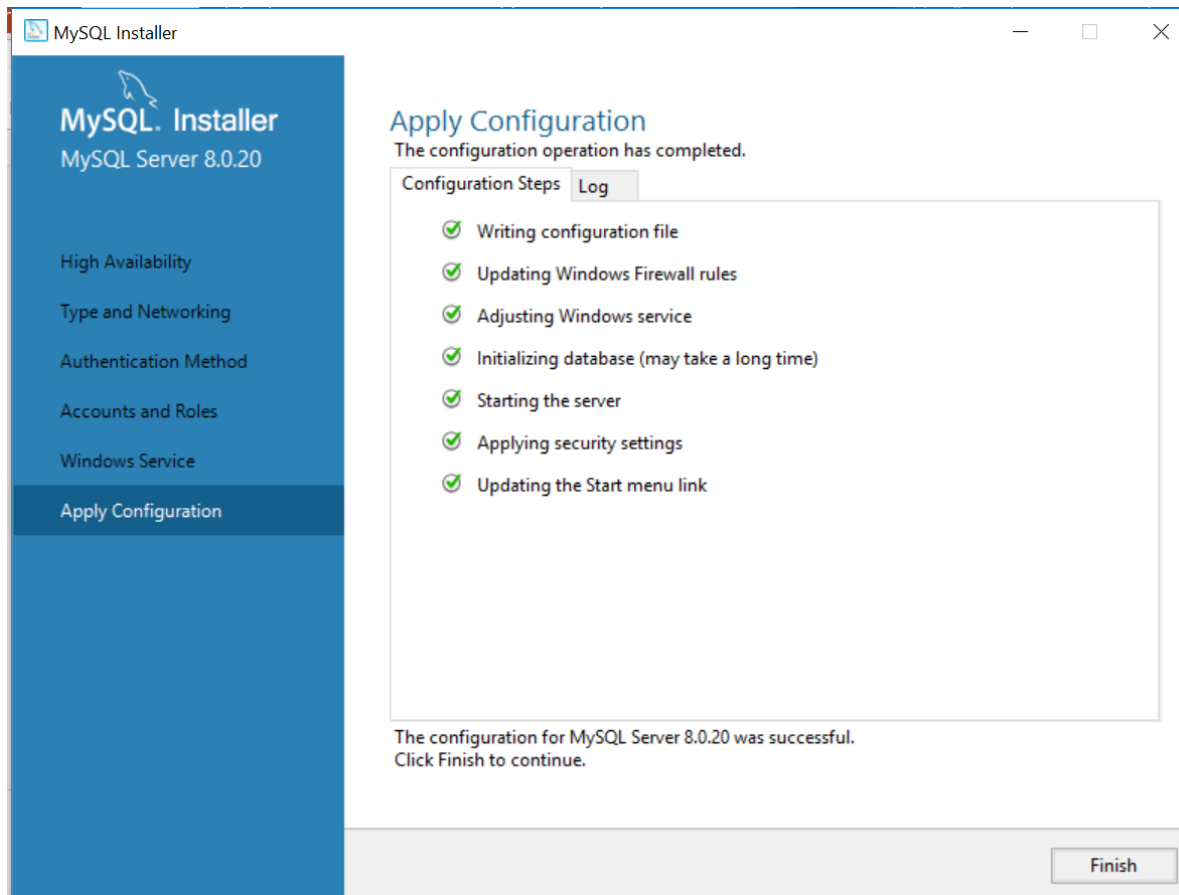
Password strength: **Weak**

MySQL User Accounts
Create MySQL user accounts for your users and applications. Assign a role to the user that consists of a set of privileges.

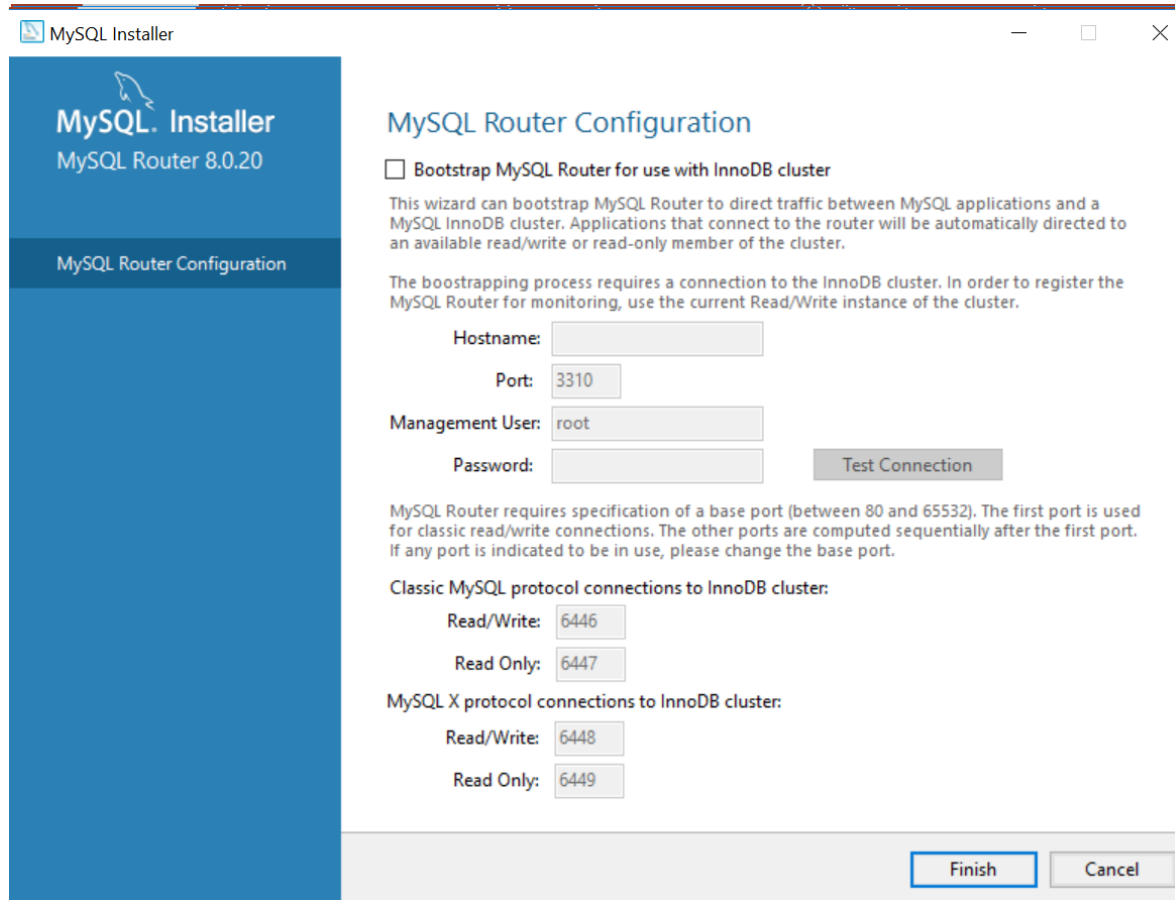
MySQL User Name	Host	User Role
-----------------	------	-----------

< Back Next > Cancel

Introduction to Data Science: MySQL installation



Introduction to Data Science: MySQL installation



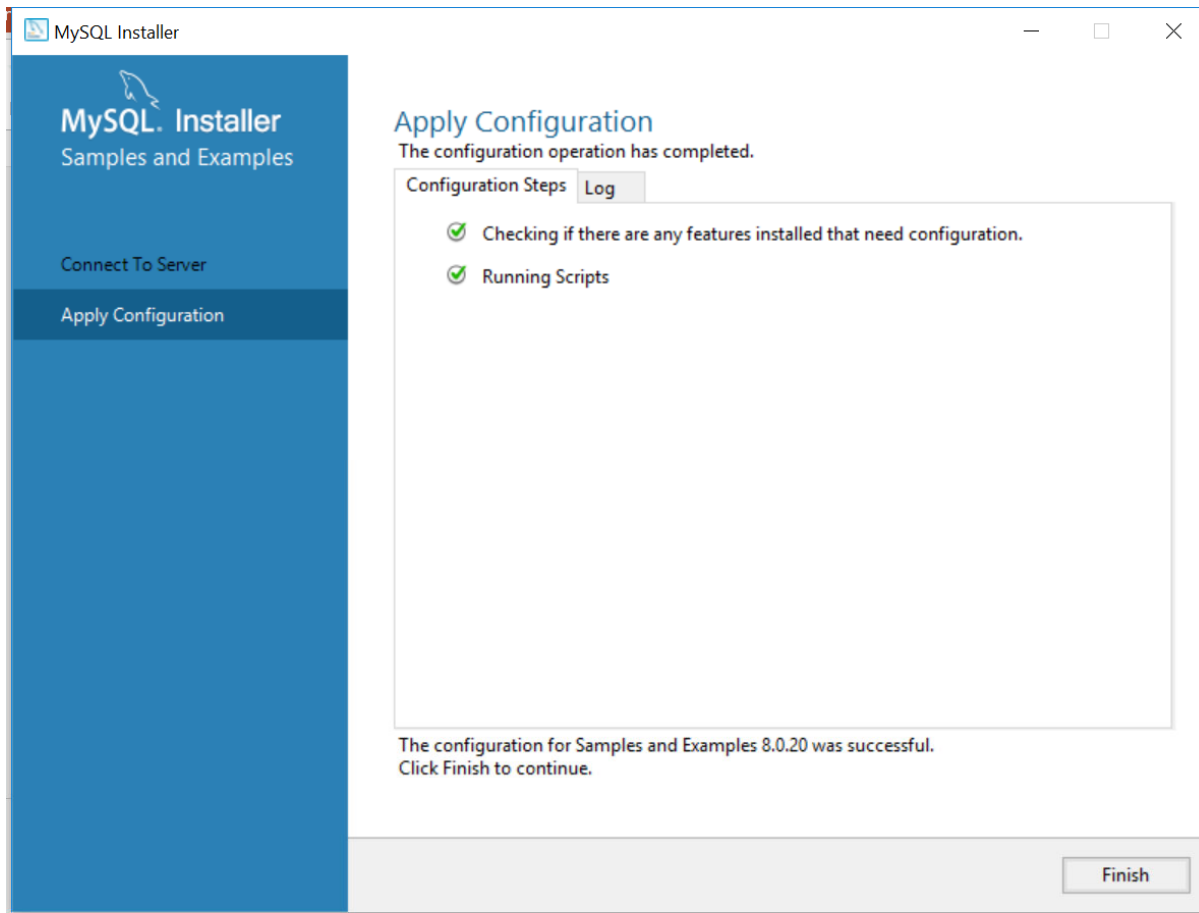
The image shows a screenshot of the MySQL Installer application window, specifically the 'MySQL Router Configuration' step. The window has a blue sidebar on the left with the MySQL logo and the text 'MySQL Installer' and 'MySQL Router 8.0.20'. The main area is white and contains the following elements:

- MySQL Router Configuration** (Section Header)
- ☐ **Bootstrap MySQL Router for use with InnoDB cluster**
- This wizard can bootstrap MySQL Router to direct traffic between MySQL applications and a MySQL InnoDB cluster. Applications that connect to the router will be automatically directed to an available read/write or read-only member of the cluster.
- The bootstrapping process requires a connection to the InnoDB cluster. In order to register the MySQL Router for monitoring, use the current Read/Write instance of the cluster.
- Hostname:
- Port:
- Management User:
- Password:
-
- MySQL Router requires specification of a base port (between 80 and 65532). The first port is used for classic read/write connections. The other ports are computed sequentially after the first port. If any port is indicated to be in use, please change the base port.
- Classic MySQL protocol connections to InnoDB cluster:

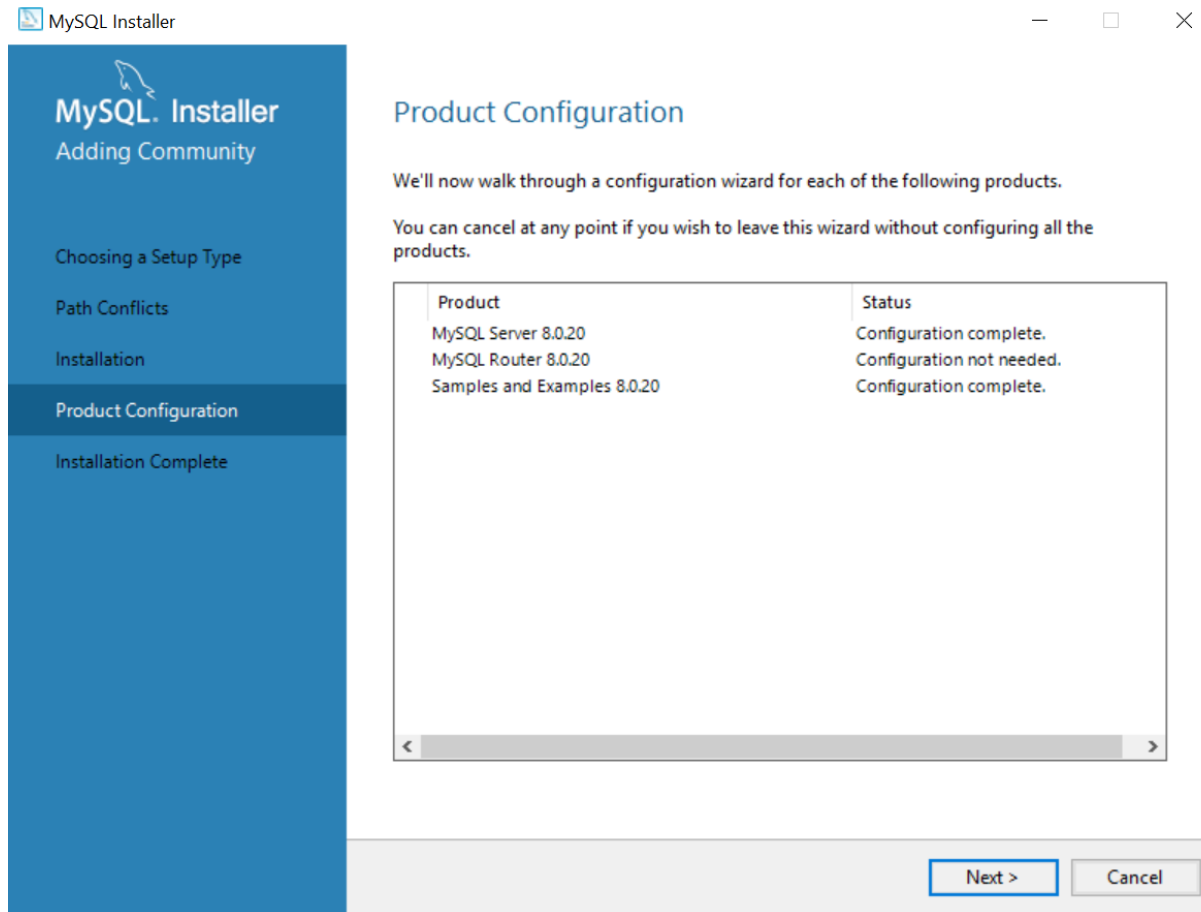
 - Read/Write:
 - Read Only:
- MySQL X protocol connections to InnoDB cluster:

 - Read/Write:
 - Read Only:
-

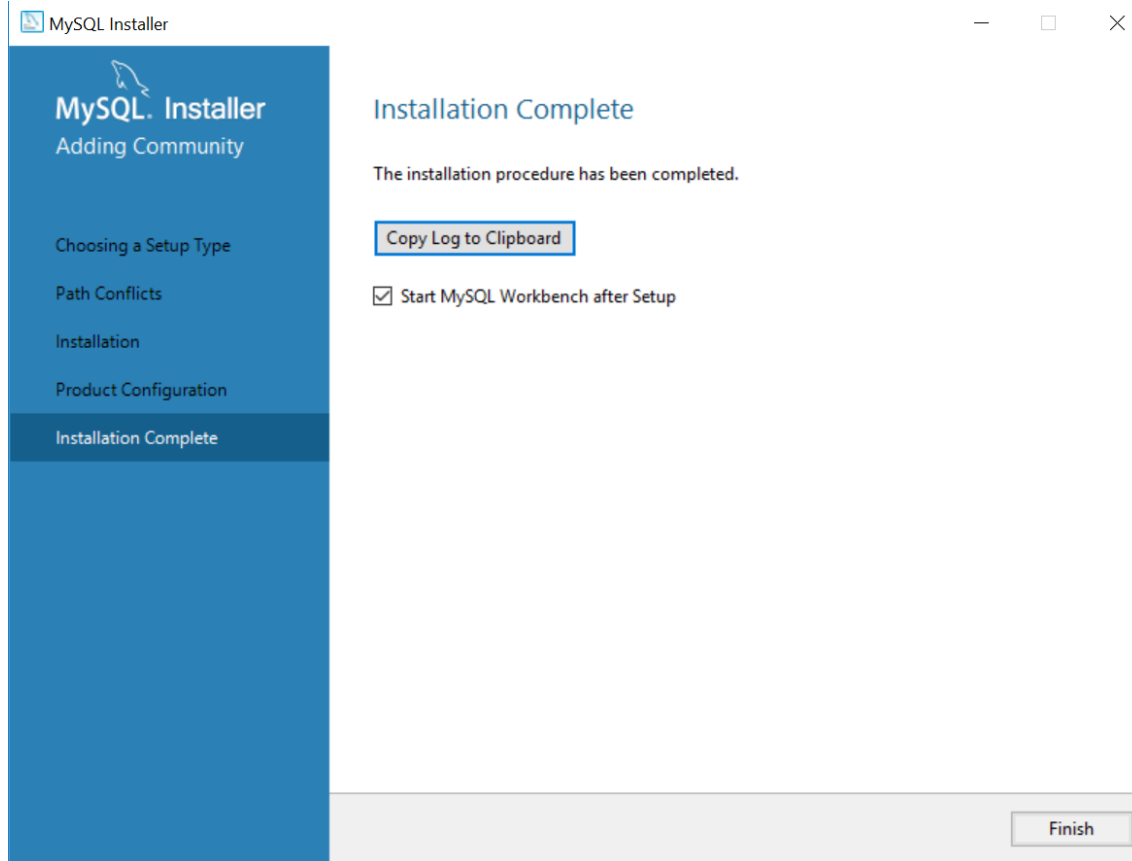
Introduction to Data Science: MySQL installation



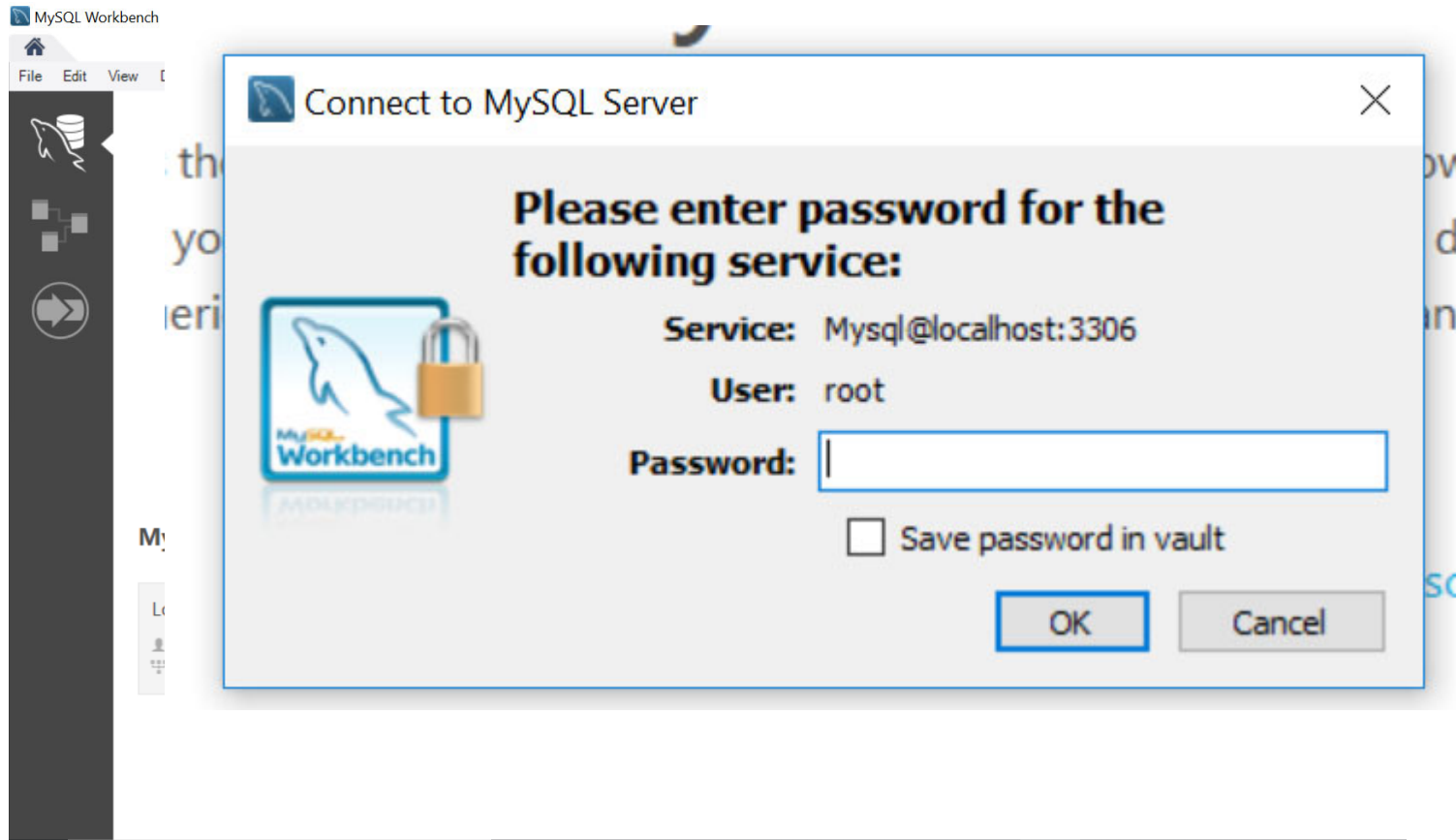
Introduction to Data Science: MySQL installation



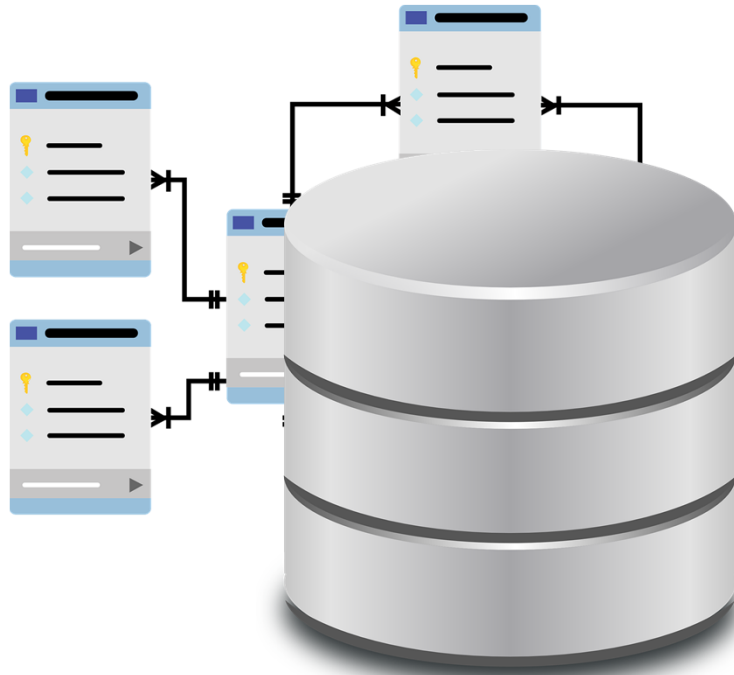
Introduction to Data Science: MySQL installation



Introduction to Data Science: MySQL installation

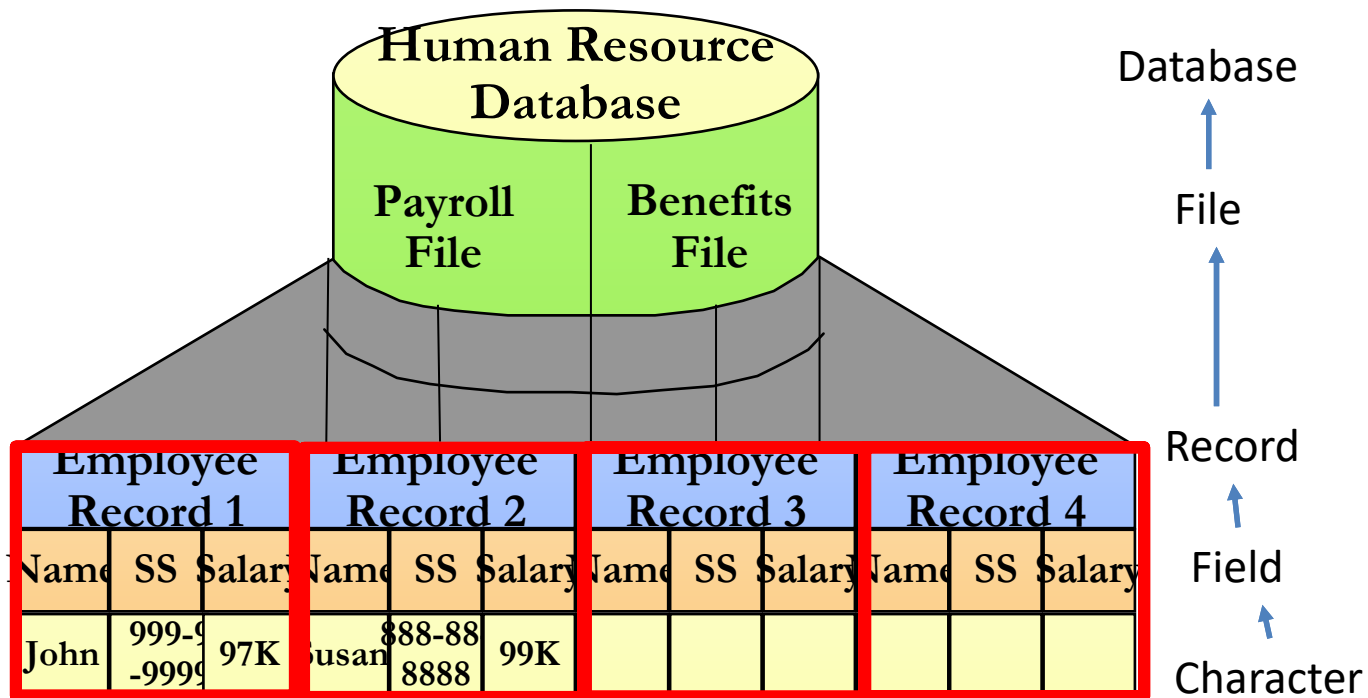


Introduction to Data Science: Structured Query Language



Structured Query Language (SQL)

Introduction to Data Science: SQL



Introduction to Data Science: SQL

Table
Name

Relation. Entity / Database Object

Employee

Employee No.	Name	SS	Salary
1	John Smith	654-45-0987	100,000
2	Susan Jones	873-54-0912	102000
3	Bob Wu	872-12-9832	150,000
4	Keith Newman	832-89-1265	76000
5	Houston Lee	236-87-4576	56,000

Record

Tuple
No. of tuples
Is called Cardinality

Field

Attributes. No. of attributes is called Degree

Introduction to Data Science: SQL

Employee

Employee No.	Name	SS	Salary
1	John Smith	654-45-0987	100,000
2	Susan Jones	873-54-0912	102000
3	Bob Wu	872-12-9832	150,000
4	Keith Newman	832-89-1265	76000
5	Houston Lee	236-87-4576	56,000

Primary Key

Surrogate key

Introduction to Data Science: SQL

Employee

Employee No.	Name	Salary
654-45-0987	John Smith	100,000
873-54-0912	Susan Jones	102000
872-12-9832	Bob Wu	150,000
832-89-1265	Keith Newman	76000
236-87-4576	Houston Lee	56,000

Primary Key

Natural key

Introduction to Data Science: SQL

Employee

Employee No.	Name	Salary	Division ID
654-45-0987	John Smith	100,000	10
873-54-0912	Susan Jones	102000	20
872-12-9832	Bob Wu	150,000	30
832-89-1265	Keith Newman	76000	20
236-87-4576	Houston Lee	56,000	40

Manager

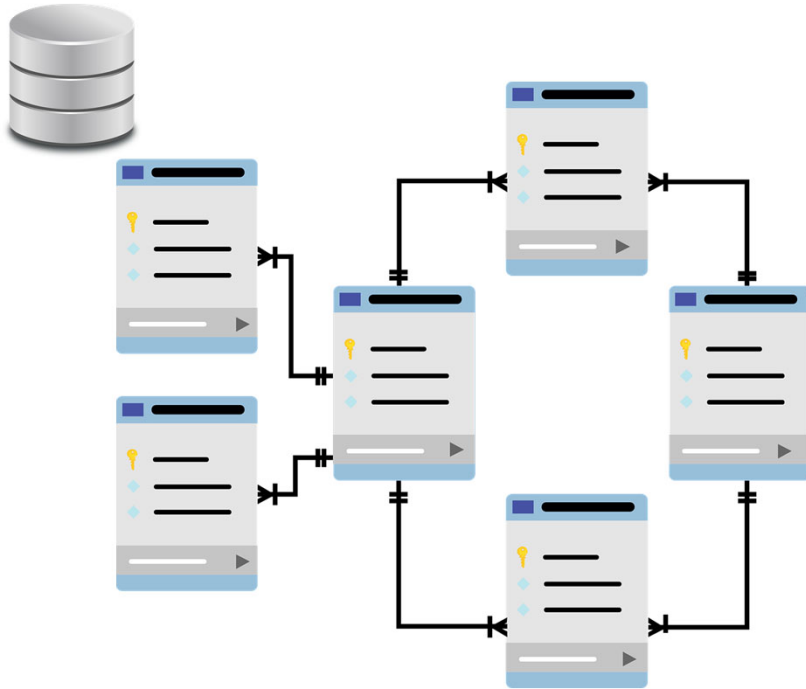
Manager ID	Name	Division ID
100	John Smith	10
200	Susan Jones	20
300	Bob Wu	20
400	Keith Newman	10
500	Houston Lee	50

Division

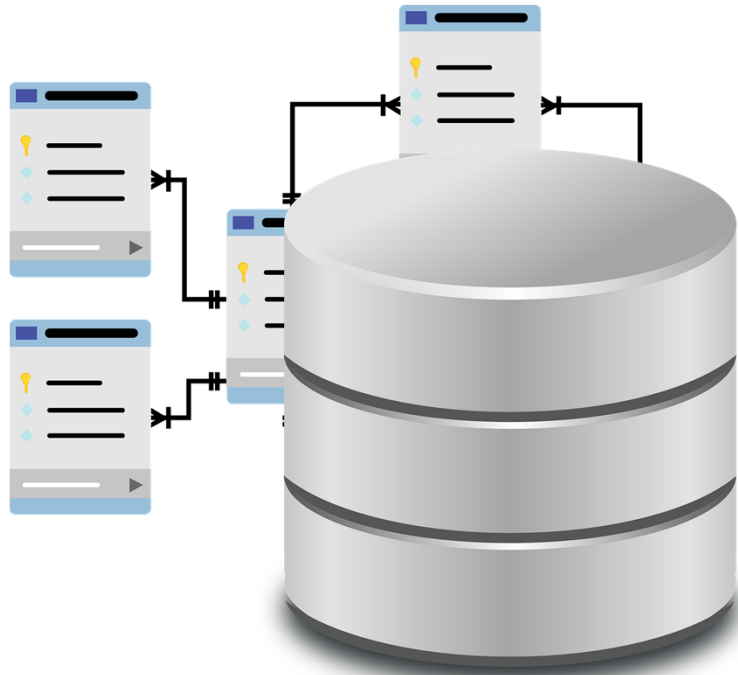
Division ID	Division Name	Budget
10	Sales	5,000,000
20	Marketing	100000000
30	Product	10,000,000,000
40	Admin	500000
50	Legal	200,000

Foreign Key

Introduction to Data Science: Structured Query Language



Introduction to Data Science: SQL DDL



Data Definition Language DDL

Introduction to Data Science: SQL DDL

- Define Database Structure
 - Create databases, tables
 - Add columns
 - Delete

Introduction to Data Science: SQL DDL

To Create a Database:

```
CREATE DATABASE myschool;
```

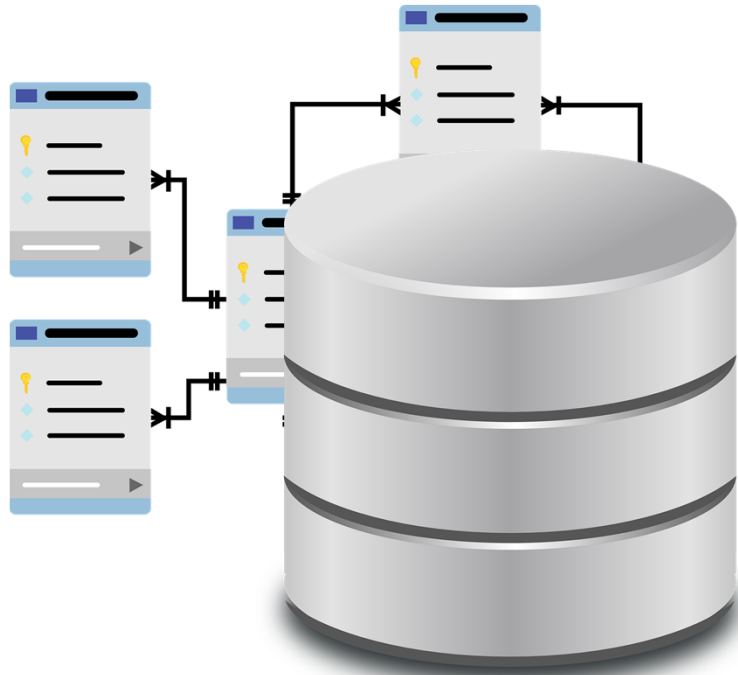
```
CREATE SCHEMA myschool;
```

Introduction to Data Science: SQL DDL

To Create a Table:

```
CREATE TABLE `myschool`.`employee` (  
  `id` INT NOT NULL, `name` VARCHAR(45)  
  NOT NULL, `salary` VARCHAR(45) NOT  
  NULL, PRIMARY KEY (`id`));
```

Introduction to Data Science: SQL Data Definition Language



Syntax and Pitfalls

Introduction to Data Science: SQL Create and Use Database

Let's create a database named "mystore"

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL

-- Create database for mystore

```
CREATE SCHEMA mystore;
```

-- Create table named inventory in mystore

```
CREATE TABLE mystore.inventory(Item_no INT NOT  
NULL, Item_name VARCHAR(100) NOT NULL,  
Unit_Price INT NOT NULL, Inventory INT, PRIMARY KEY  
(Item_no));
```

-- Populate table with values/data

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (2321, 'Dell Laptop',  
1500, 56);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (5432, 'Seagate Drive',  
200, 100);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (5674, 'Kingston USB  
Drive', 70, 500);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (8542, 'Backpack', 100,  
45);
```

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL DDL

-- Select database to use

USE mystore;

/* once USE mystore is executed, we can eliminate the dot operator and database name */

CREATE TABLE inventory(Item_no **INT NOT NULL**,
Item_name **VARCHAR(100) NOT NULL**, Unit_Price **INT NOT NULL**, Inventory **INT**, **PRIMARY KEY** (Item_no));

-- Insert new product with null value

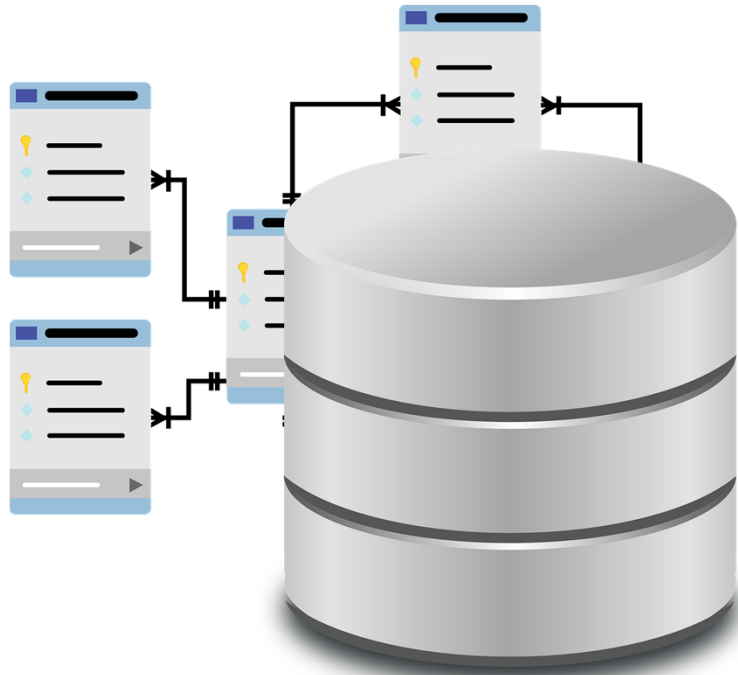
INSERT INTO inventory (Item_no, Item_name, Unit_Price, Inventory) **VALUES** (2348, 'HP Laptop', 1000, null);

-- Insert new product with no null columns only

INSERT INTO inventory (Item_no, Item_name, Unit_Price) **VALUES** (7344, 'Lenovo Laptop', 988);

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL Data Definition Language



Syntax and Pitfalls

Introduction to Data Science: SQL DDL

-- Select database to use

USE mystore;

/* once USE mystore is executed, we can eliminate the dot operator and database name */

CREATE TABLE inventory(Item_no **INT NOT NULL**,
Item_name **VARCHAR(100) NOT NULL**, Unit_Price **INT NOT NULL**, Inventory **INT**, **PRIMARY KEY** (Item_no));

-- Insert new product with null value

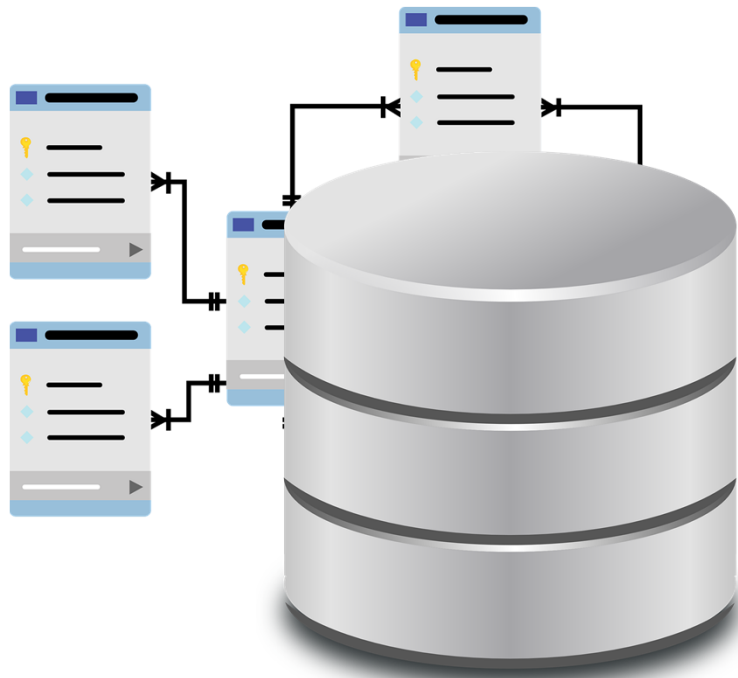
INSERT INTO inventory (Item_no, Item_name,
Unit_Price, Inventory) **VALUES** (2348, 'HP Laptop', 1000,
null);

-- Insert new product with no null columns only

INSERT INTO inventory (Item_no, Item_name,
Unit_Price) **VALUES** (7344, 'Lenovo Laptop', 988);

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL Build Query Script



Build Query Script on MySQL Workbench

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL

-- Create database for mystore

```
CREATE SCHEMA mystore;
```

-- Create table named inventory in mystore

```
CREATE TABLE mystore.inventory(Item_no INT NOT  
NULL, Item_name VARCHAR(100) NOT NULL,  
Unit_Price INT NOT NULL, Inventory INT, PRIMARY KEY  
(Item_no));
```

-- Populate table with values/data

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (2321, 'Dell Laptop',  
1500, 56);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (5432, 'Seagate Drive',  
200, 100);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (5674, 'Kingston USB  
Drive', 70, 500);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (8542, 'Backpack', 100,  
45);
```

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL DDL

-- Select database to use

USE mystore;

/* once USE mystore is executed, we can eliminate the dot operator and database name */

CREATE TABLE inventory(Item_no **INT NOT NULL**,
Item_name **VARCHAR(100) NOT NULL**, Unit_Price **INT NOT NULL**, Inventory **INT**, **PRIMARY KEY** (Item_no));

-- Insert new product with null value

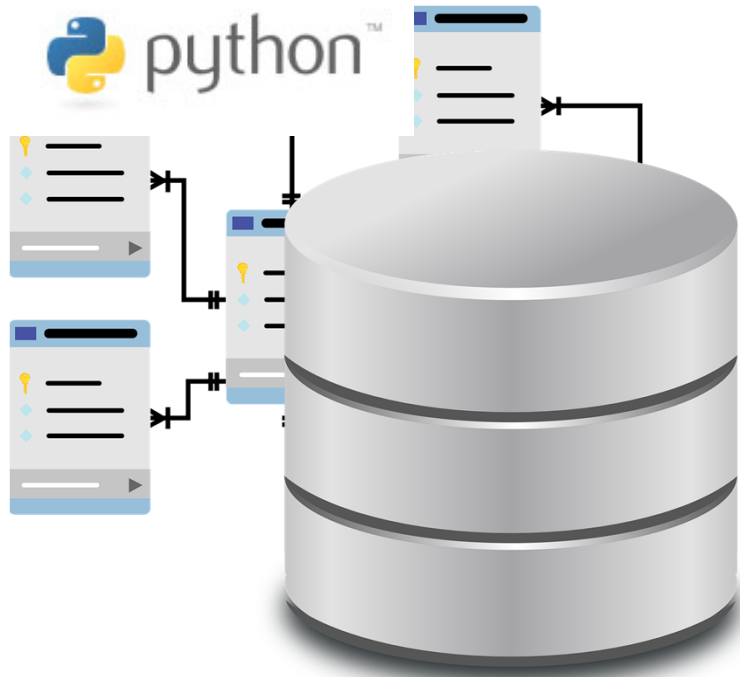
INSERT INTO inventory (Item_no, Item_name, Unit_Price, Inventory) **VALUES** (2348, 'HP Laptop', 1000, null);

-- Insert new product with no null columns only

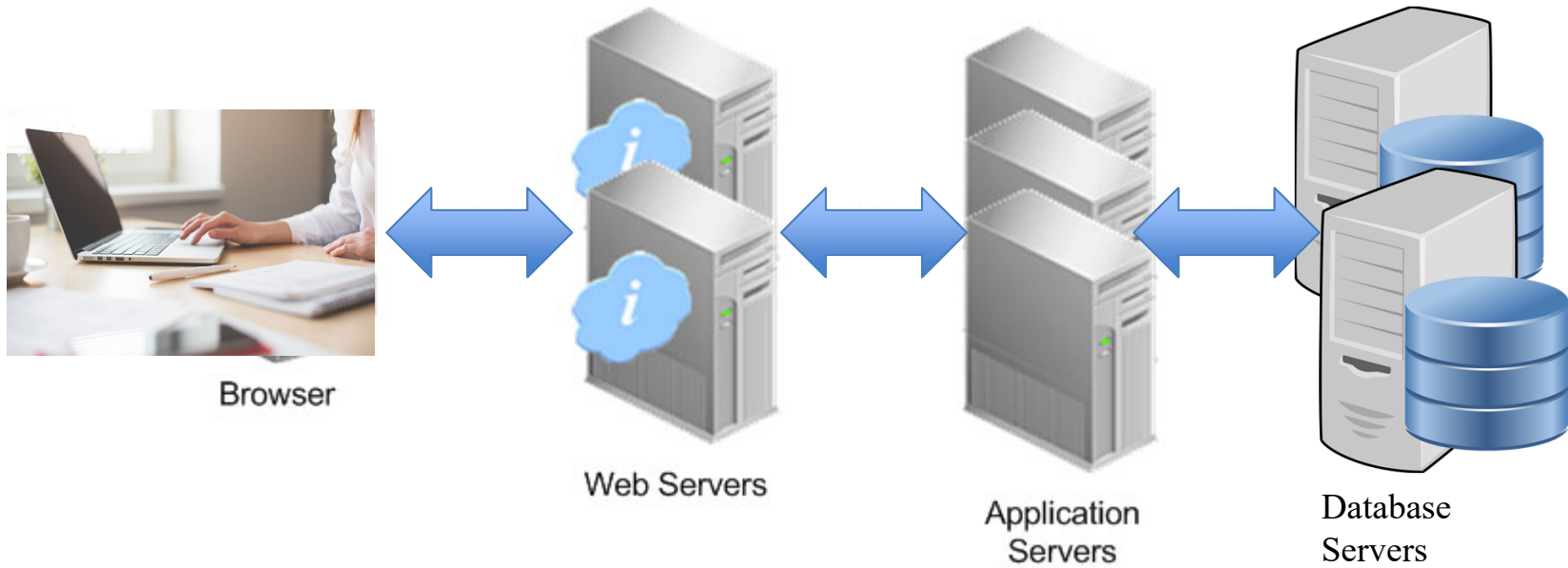
INSERT INTO inventory (Item_no, Item_name, Unit_Price) **VALUES** (7344, 'Lenovo Laptop', 988);

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Data Science: Python and MySQL Integration



Data Science: Python and MySQL Integration



Data Science: Python and MySQL Integration

```
import mysql.connector as sq
```

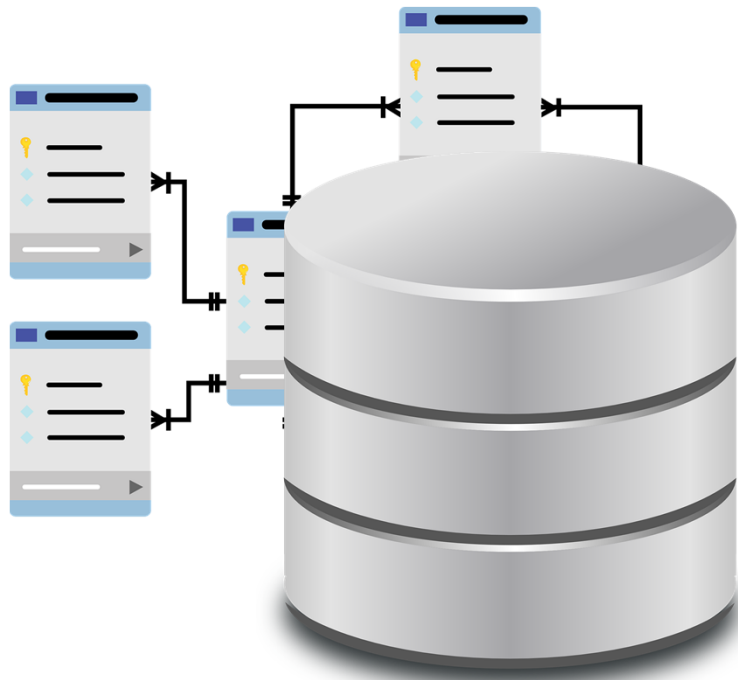
```
mydb=sq.connect(host="localhost",user="root",passwd="ucla", buffered=True)
```

```
mycursor = mydb.cursor()
```

```
mycursor.execute("SHOW DATABASES")
```

```
for db in mycursor:  
    print (db)
```

SQL Create and Insert using Executemany



Create and Insert executemany

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45


```
import mysql.connector as sq
```

```
mydb=sq.connect(host="localhost",user="root",passwd="ucla", buffered=True)
```

```
mycursor = mydb.cursor()
```

```
mycursor.execute("CREATE SCHEMA mystore")
```

```
mycursor.execute("CREATE TABLE mystore.inventory(Item_no INT NOT NULL, Item_name VARCHAR(100) NOT NULL, \nUnit_Price INT NOT NULL, Inventory INT, PRIMARY KEY (Item_no))")
```

```
mycursor.execute("INSERT INTO mystore.inventory (Item_no, Item_name, Unit_Price, Inventory) \nVALUES (2321, 'Dell Laptop', 1500, 56)")
```

```
mycursor.execute("INSERT INTO mystore.inventory (Item_no, Item_name, Unit_Price, Inventory) \nVALUES (5432, 'Seagate Drive', 200, 100)")
```

```
mycursor.execute("INSERT INTO mystore.inventory (Item_no, Item_name, Unit_Price, Inventory) \nVALUES (5674, 'Kingston USB Drive', 70, 500)")
```

```
mycursor.execute("INSERT INTO mystore.inventory (Item_no, Item_name, Unit_Price, Inventory) \nVALUES (8542, 'Backpack', 100, 45)")
```

```
mydb.commit()
```

```
import mysql.connector as sq
```

```
mydb=sq.connect(host="localhost",user="root",passwd="ucla", buffered=True)
```

```
mycursor = mydb.cursor()
```

```
mycursor.execute("CREATE SCHEMA mystore")
```

```
mycursor.execute("CREATE TABLE mystore.inventory(Item_no INT NOT NULL, Item_name VARCHAR(100) NOT NULL, \\\nUnit_Price INT NOT NULL, Inventory INT, PRIMARY KEY (Item_no))")
```

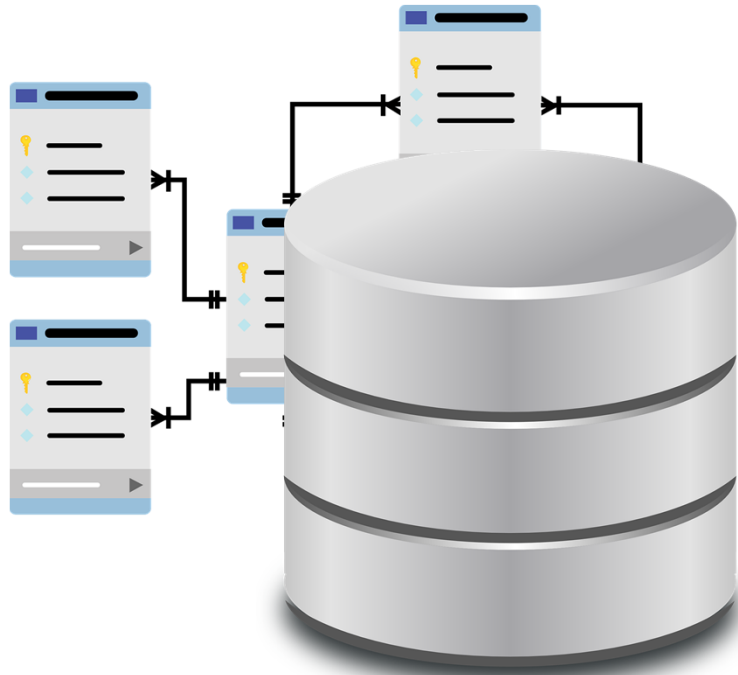
```
SQLCMD = "INSERT INTO PyDB.PyTABLE (id, name, salary)VALUES(%s,%s,%s)"
```

```
values = [(100,'John Hall',12300.55),(101,'John Smith',10300.15),(102,'Susan Jones',11500.57)]
```

```
mycursor.executemany(SQLCMD, values)
```

```
mydb.commit()
```

SQL Data Definition Language



Create and Insert

SQL Create and Use Database

Let's create a database named "mystore"

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL

-- Create database for mystore

```
CREATE SCHEMA mystore;
```

-- Create table named inventory in mystore

```
CREATE TABLE mystore.inventory(Item_no INT NOT  
NULL, Item_name VARCHAR(100) NOT NULL,  
Unit_Price INT NOT NULL, Inventory INT, PRIMARY KEY  
(Item_no));
```

-- Populate table with values/data

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (2321, 'Dell Laptop',  
1500, 56);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (5432, 'Seagate Drive',  
200, 100);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (5674, 'Kingston USB  
Drive', 70, 500);
```

```
INSERT INTO mystore.inventory (Item_no, Item_name,  
Unit_Price, Inventory) VALUES (8542, 'Backpack', 100,  
45);
```

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

Introduction to Data Science: SQL DDL

-- Select database to use

USE mystore;

/* once USE mystore is executed, we can eliminate the dot operator and database name */

CREATE TABLE inventory(Item_no **INT NOT NULL**,
Item_name **VARCHAR(100) NOT NULL**, Unit_Price **INT NOT NULL**, Inventory **INT**, **PRIMARY KEY** (Item_no));

-- Insert new product with null value

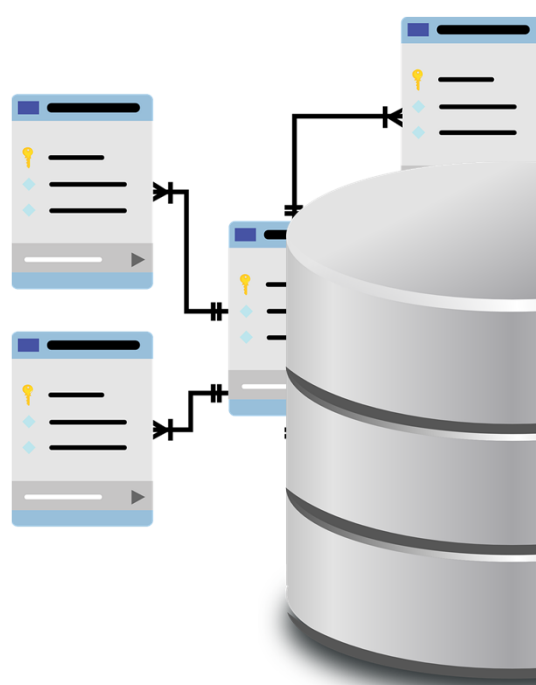
INSERT INTO inventory (Item_no, Item_name, Unit_Price, Inventory) **VALUES** (2348, 'HP Laptop', 1000, null);

-- Insert new product with no null columns only

INSERT INTO inventory (Item_no, Item_name, Unit_Price) **VALUES** (7344, 'Lenovo Laptop', 988);

Item_no	Item_name	Unit_Price	Inventory
2321	Dell Laptop	1500	56
5432	Seagate Drive	200	100
5674	Kingston USB Drive	70	500
8542	Backpack	100	45

SQL Load Data from CSV File



The diagram illustrates the process of loading data from CSV files into a database. On the left, there are three server icons representing CSV files. Arrows point from these files to a central database cylinder, indicating the data load process. The database cylinder is connected to a table on the right.

	A	B	C	D	E	F	G
1	Sales_Date	Day_of_W	Salesman	Temperatu	Tweets	Price	Sales
2	1/1/2019	Tuesday	John	72	2	0.5	177
3	1/2/2019	Wednesd	John	82	3	0.5	127
4	1/3/2019	Thursday	John	69	5	0.5	172
5	1/4/2019	Friday	John	100	7	0.5	150
6	1/5/2019	Saturday	Ada	69	6	0.3	103
7	1/6/2019	Sunday	Ada	91	8	0.5	120
8	1/7/2019	Monday	Ada	81	3	0.3	96
9	1/8/2019	Tuesday	John	88	6	0.5	150
10	1/9/2019	Wednesd	John	69	8	0.5	177
11	1/10/2019	Thursday	John	61	10	0.5	190
12	1/11/2019	Friday	John	79	1	0.5	154
13	1/12/2019	Saturday	John	94	9	0.5	160
14	1/13/2019	Sunday	John	80	5	0.5	161
15	1/14/2019	Monday	John	64	8	0.5	185
16	1/15/2019	Tuesday	Ada	94	6	0.5	137
17	1/16/2019	Wednesd	Ada	68	6	0.3	106
18	1/17/2019	Thursday	Ada	74	2	0.3	85
19	1/18/2019	Friday	John	89	4	0.5	122
20	1/19/2019	Saturday	John	87	6	0.5	187
21	1/20/2019	Sunday	John	100	7	0.5	165
22	1/21/2019	Monday	John	89	8	0.5	158

SQL Load Data from CSV File

 Cookies Sample - Notepad

File Edit Format View Help

Sales_Date,Day_of_Week,Salesman,Temperature,Tweets,Price,Sales

1/1/2019,Tuesday,John,72,2,0.5,177

1/2/2019,Wednesday,John,82,3,0.5,127

1/3/2019,Thursday,John,69,5,0.5,172

1/4/2019,Friday,John,100,7,0.5,150

1/5/2019,Saturday,Ada,69,6,0.3,103

1/6/2019,Sunday,Ada,91,8,0.5,120

1/7/2019,Monday,Ada,81,3,0.3,96

1/8/2019,Tuesday,John,88,6,0.5,150

1/9/2019,Wednesday,John,69,8,0.5,177

1/10/2019,Thursday,John,61,10,0.5,190

1/11/2019,Friday,John,79,1,0.5,154

1/12/2019,Saturday,John,94,9,0.5,160

1/13/2019,Sunday,John,80,5,0.5,161

1/14/2019,Monday,John,64,8,0.5,185

1/15/2019,Tuesday,Ada,94,6,0.5,137

1/16/2019,Wednesday,Ada,68,6,0.3,106

1/17/2019,Thursday,Ada,74,2,0.3,85

1/18/2019,Friday,John,89,4,0.5,122

1/19/2019,Saturday,John,87,6,0.5,187

1/20/2019,Sunday,John,100,7,0.5,165

1/21/2019,Monday,John,89,8,0.5,158

1/22/2019,Tuesday,John,82,6,0.5,192

1/23/2019,Wednesday,Ada,60,7,0.3,87

<

SQL Load Data from CSV File

Step 1: Understand the structure (column heading, data type)

Step 2: Create SQL script

- Create database
- Create table with the same structure observed from CSV file
- Load data from CSV file

Step 3: Run SQL script

SQL Load Data from CSV File

```
CREATE SCHEMA cookies;
```

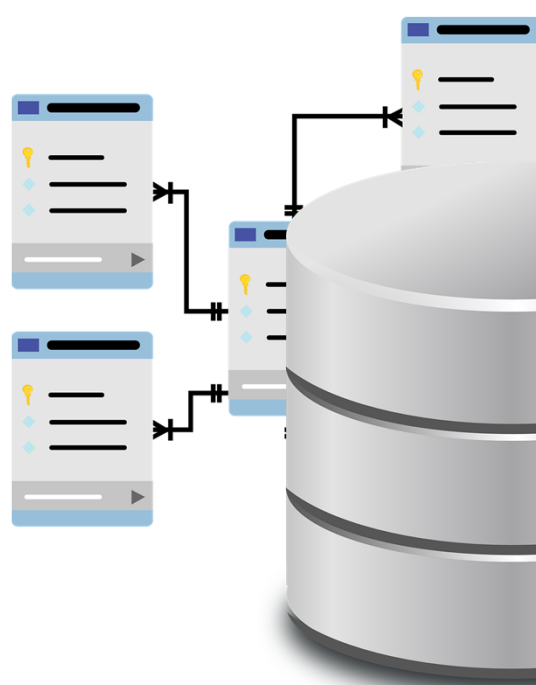
```
CREATE TABLE cookies.sales  
(Sales_Date varchar(10),  
Day_of_Week varchar(10),  
Salesman varchar(10),  
Temperature INT,  
Tweets INT,  
Price INT,  
Sales INT);
```

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Cookies Sample.csv'  
INTO TABLE cookies.sales  
FIELDS TERMINATED BY ','  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

SQL Load Data from CSV File

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
▶	1/1/2019	Tuesday	John	72	2	1	177
	1/2/2019	Wednesday	John	82	3	1	127
	1/3/2019	Thursday	John	69	5	1	172
	1/4/2019	Friday	John	100	7	1	150
	1/5/2019	Saturday	Ada	69	6	0	103
	1/6/2019	Sunday	Ada	91	8	1	120

Python MySQL Load Data from CSV File



The diagram illustrates the process of loading data from CSV files into a MySQL database. On the left, there are three server icons representing CSV files. Arrows indicate the data flow from these files into a central MySQL database cylinder. The database is connected to a table containing 22 rows of data.

	A	B	C	D	E	F	G
1	Sales_Date	Day_of_W	Salesman	Temperatu	Tweets	Price	Sales
2	1/1/2019	Tuesday	John	72	2	0.5	177
3	1/2/2019	Wednesd	John	82	3	0.5	127
4	1/3/2019	Thursday	John	69	5	0.5	172
5	1/4/2019	Friday	John	100	7	0.5	150
6	1/5/2019	Saturday	Ada	69	6	0.3	103
7	1/6/2019	Sunday	Ada	91	8	0.5	120
8	1/7/2019	Monday	Ada	81	3	0.3	96
9	1/8/2019	Tuesday	John	88	6	0.5	150
10	1/9/2019	Wednesd	John	69	8	0.5	177
11	1/10/2019	Thursday	John	61	10	0.5	190
12	1/11/2019	Friday	John	79	1	0.5	154
13	1/12/2019	Saturday	John	94	9	0.5	160
14	1/13/2019	Sunday	John	80	5	0.5	161
15	1/14/2019	Monday	John	64	8	0.5	185
16	1/15/2019	Tuesday	Ada	94	6	0.5	137
17	1/16/2019	Wednesd	Ada	68	6	0.3	106
18	1/17/2019	Thursday	Ada	74	2	0.3	85
19	1/18/2019	Friday	John	89	4	0.5	122
20	1/19/2019	Saturday	John	87	6	0.5	187
21	1/20/2019	Sunday	John	100	7	0.5	165
22	1/21/2019	Monday	John	89	8	0.5	158

Python MySQL Load Data from CSV File

 Cookies Sample - Notepad

File Edit Format View Help

Sales_Date,Day_of_Week,Salesman,Temperature,Tweets,Price,Sales

1/1/2019,Tuesday,John,72,2,0.5,177

1/2/2019,Wednesday,John,82,3,0.5,127

1/3/2019,Thursday,John,69,5,0.5,172

1/4/2019,Friday,John,100,7,0.5,150

1/5/2019,Saturday,Ada,69,6,0.3,103

1/6/2019,Sunday,Ada,91,8,0.5,120

1/7/2019,Monday,Ada,81,3,0.3,96

1/8/2019,Tuesday,John,88,6,0.5,150

1/9/2019,Wednesday,John,69,8,0.5,177

1/10/2019,Thursday,John,61,10,0.5,190

1/11/2019,Friday,John,79,1,0.5,154

1/12/2019,Saturday,John,94,9,0.5,160

1/13/2019,Sunday,John,80,5,0.5,161

1/14/2019,Monday,John,64,8,0.5,185

1/15/2019,Tuesday,Ada,94,6,0.5,137

1/16/2019,Wednesday,Ada,68,6,0.3,106

1/17/2019,Thursday,Ada,74,2,0.3,85

1/18/2019,Friday,John,89,4,0.5,122

1/19/2019,Saturday,John,87,6,0.5,187

1/20/2019,Sunday,John,100,7,0.5,165

1/21/2019,Monday,John,89,8,0.5,158

1/22/2019,Tuesday,John,82,6,0.5,192

1/23/2019,Wednesday,Ada,60,7,0.3,87

<

```
import mysql.connector as sq
mydb=sq.connect(host="localhost",user="root",passwd="ucla", buffered=True)
```

```
mycursor = mydb.cursor()
mycursor.execute('CREATE SCHEMA cookies')
```

```
SQLCMD = 'CREATE TABLE cookies.sales (Sales_Date varchar(10), \
Day_of_Week varchar(10), Salesman varchar(10), Temperature INT, \
Tweets INT, Price FLOAT, Sales INT)'
```

```
mycursor.execute(SQLCMD)
```

```
SQLCMD = "LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Cookies Sample.csv' \
INTO TABLE cookies.sales FIELDS TERMINATED BY ',' LINES TERMINATED BY '\\n' IGNORE 1 ROWS"
```

```
mycursor.execute(SQLCMD)
mydb.commit()
```

Python MySQL Load Data from CSV File

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
▶	1/1/2019	Tuesday	John	72	2	1	177
	1/2/2019	Wednesday	John	82	3	1	127
	1/3/2019	Thursday	John	69	5	1	172
	1/4/2019	Friday	John	100	7	1	150
	1/5/2019	Saturday	Ada	69	6	0	103
	1/6/2019	Sunday	Ada	91	8	1	120

Python Pandas DataFrame



DataFrame is a 2-dimensional labeled data structure with columns of potentially different types

Data Science: Python Pandas DataFrame

	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

Python Pandas DataFrame

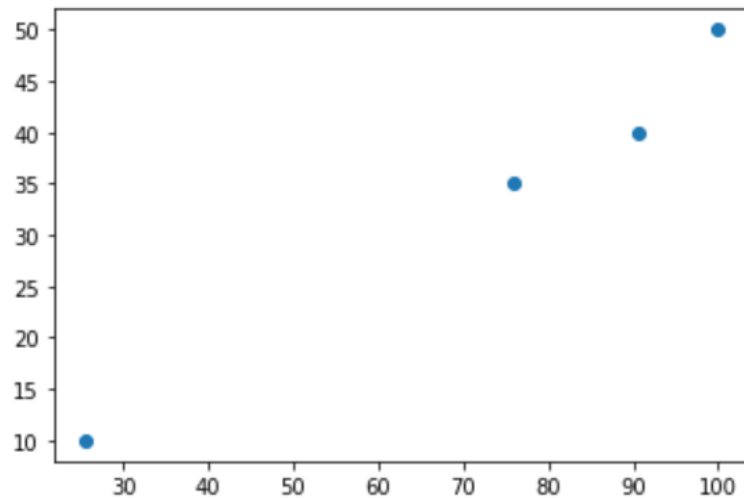
```
import pandas as pd
from matplotlib import pyplot as plt

grades = [90.5, 100.0, 75.8, 25.6]
studytime = [40, 50, 35, 10]
# Convert the List of Grades into Excel Spreadsheet Lookalike Column Format
# Use Pandas DataFrame to convert ONE List to start with (One Column)
df = pd.DataFrame(grades, columns = ["Grades"])
# Now we can add another List as second column to the dataframe created
df["Studytime"] = studytime
# Take a Look at the dataframe. Doesn't it Look just like Excel?
print(df)
# Use Pandas DataFrame Correlation to perform Pearson R
print(df.corr())
plt.scatter(grades, studytime)
plt.show()
```

Data Science: Python Pandas DataFrame and Correlation

	Grades	Studytime
0	90.5	40
1	100.0	50
2	75.8	35
3	25.6	10

	Grades	Studytime
Grades	1.00000	0.99219
Studytime	0.99219	1.00000



Data Science: Python Pandas DataFrame and Correlation Demo

Python Pandas DataFrame

```
import pandas as pd
from matplotlib import pyplot as plt
%matplotlib inline

grades = [90.5, 100.0, 75.8, 25.6]
studytime = [40, 50, 35, 10]

# Convert the List of Grades into Excel Spreadsheet Lookalike Column Format
data = list(zip(grades, studytime))

df = pd.DataFrame(data, columns = ["Grades", "StudyTime"])

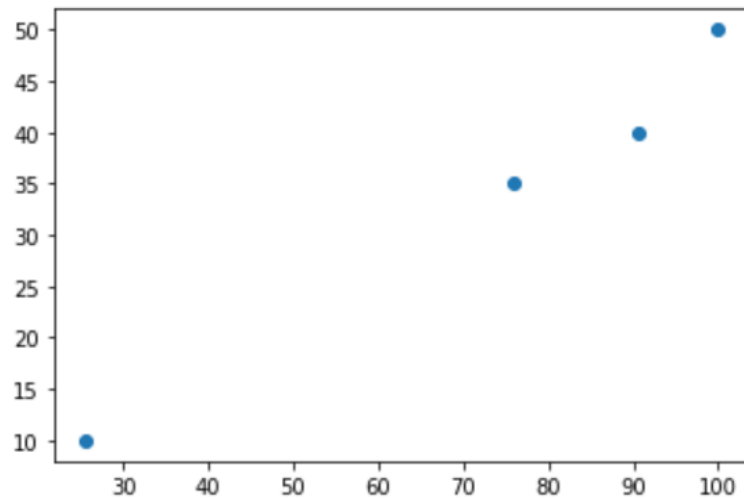
print (df)

# Use Pandas DataFrame Correlation
print (df.corr())
plt.scatter(studytime, grades)
plt.show()
```

Data Science: Python Pandas DataFrame and Correlation

	Grades	Studytime
0	90.5	40
1	100.0	50
2	75.8	35
3	25.6	10

	Grades	Studytime
Grades	1.00000	0.99219
Studytime	0.99219	1.00000



Python Pandas DataFrame

```
import pandas as pd
```

```
MyClass = {'students':['Bruce', 'Jane', 'Nancy',  
                        'Bill'],  
           'grades':[10, 9, 9, 8]}
```

```
df = pd.DataFrame(MyClass)
```

	students	grades
0	Bruce	10
1	Jane	9
2	Nancy	9
3	Bill	8

Python Pandas DataFrame

```
import pandas as pd
```

```
MyClass = {'students':['Bruce', 'Jane', 'Nancy',  
                        'Bill'],  
           'grades':[10, 9, 9, 8]}
```

```
df = pd.DataFrame(MyClass, index  
                  =["ID1","ID2","ID3","ID4"])
```

	students	grades
ID1	Bruce	10
ID2	Jane	9
ID3	Nancy	9
ID4	Bill	8

Python Pandas DataFrame

```
import pandas as pd
```

```
MyClass =
```

```
{'John':10,'Jake':9,'Jackie':8,'Jack':7,'Jane':6,'Jo':10,'Ja':9,'Jac':8,'Jacky':7,'Jan':6}
```

```
df = pd.DataFrame(MyClass, index=[1, 2, 3])
```

	John	Jake	Jackie	Jack	Jane	Jo	Ja	Jac	Jacky	Jan
1	10	9	8	7	6	10	9	8	7	6
2	10	9	8	7	6	10	9	8	7	6
3	10	9	8	7	6	10	9	8	7	6

Python Pandas DataFrame

```
MyInventory = {  
    "Item": ["coffee", "chocolate", "tea",  
"water"],  
    "Promotion": [False, False, True, False],  
    "Price": [5.95, 5.95, 3.95, 2.95],  
    "Stock": [100, 250, 1000, 1200]  
}
```

```
ddf = pd.DataFrame(MyInventory)  
ddf
```

	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250
2	tea	True	3.95	1000
3	water	False	2.95	1200

Python Pandas DataFrame

```
Inv2 = {  
    "Item": ["coffee", "chocolate", "tea",  
            "water"],  
    "Promotion": ["no", "no", "yes", "yes"],  
    "Price": [5.95, 5.95, 3.95, 2.95],  
    "Stock": [100, 250, 1000, 1200]  
}
```

```
InvDF = pd.DataFrame(Inv2)
```

```
InvDF
```

	Item	Promotion	Price	Stock
0	coffee	no	5.95	100
1	chocolate	no	5.95	250
2	tea	yes	3.95	1000
3	water	yes	2.95	1200

Python Pandas DataFrame

```
Inv2 = {  
    "Item": ["coffee", "chocolate", "tea",  
            "water"],  
    "Promotion": ["no", "no", "yes", "yes"],  
    "Price": [5.95, 5.95, 3.95, 2.95],  
    "Stock": [100, 250, 1000, 1200]  
}
```

```
InvDF = pd.DataFrame(Inv2)
```

```
InvDF
```

```
InvDF = InvDF.replace({'Promotion': {'no':  
False, 'yes': True}})
```

	Item	Promotion	Price	Stock
0	coffee	no	5.95	100
1	chocolate	no	5.95	250
2	tea	yes	3.95	1000
3	water	yes	2.95	1200

	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250
2	tea	True	3.95	1000
3	water	True	2.95	1200

Python Pandas DataFrame

```
InvDF[InvDF["Promotion"] == False]
```

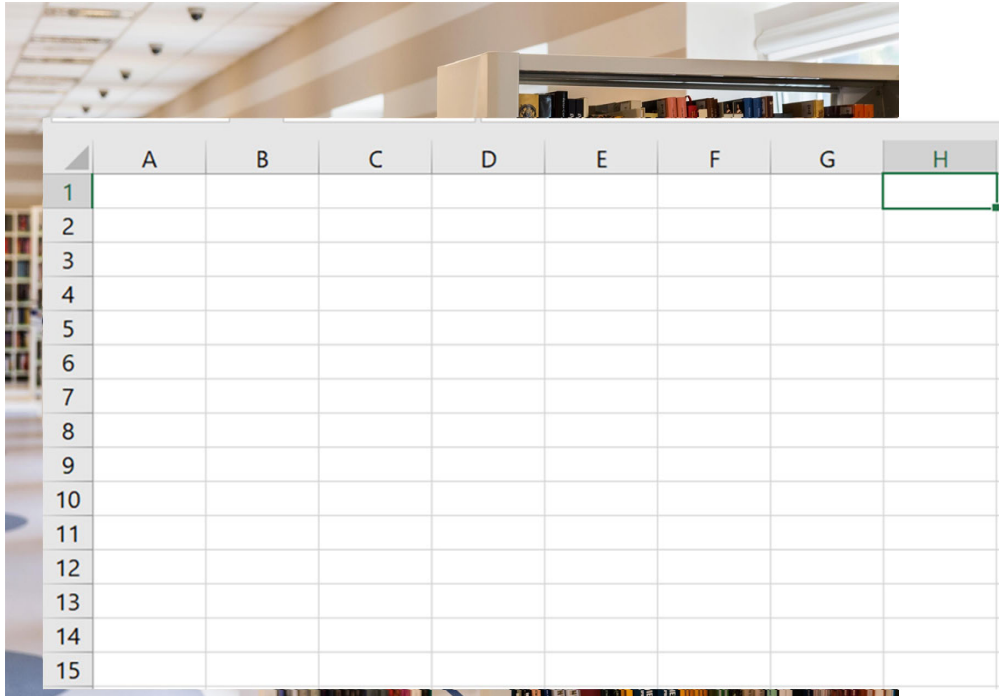
	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250

```
InvDF[InvDF["Price"] < 5]
```

	Item	Promotion	Price	Stock
2	tea	True	3.95	1000
3	water	True	2.95	1200

	Item	Promotion	Price	Stock
0	coffee	False	5.95	100
1	chocolate	False	5.95	250
2	tea	True	3.95	1000
3	water	True	2.95	1200

Data Science: Python Pandas DataFrame from Excel

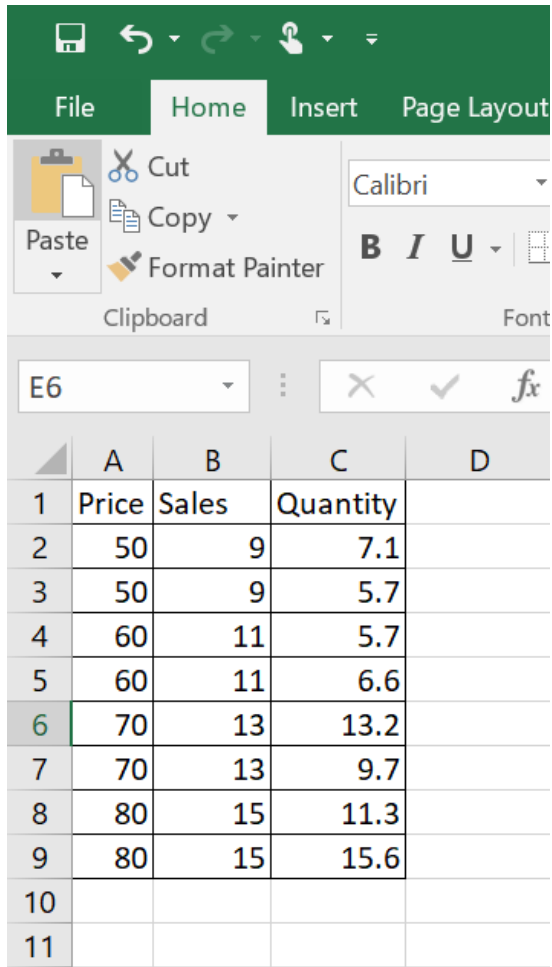


	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

Pandas **DataFrame** Review

DataFrame is a 2-dimensional labeled data structure with columns of potentially different types

Data Science: Python Pandas DataFrame from Excel



The screenshot shows the Microsoft Excel interface. The ribbon is set to the 'Home' tab. The font is 'Calibri'. The cell 'E6' is selected. The spreadsheet contains the following data:

	A	B	C	D
1	Price	Sales	Quantity	
2	50	9	7.1	
3	50	9	5.7	
4	60	11	5.7	
5	60	11	6.6	
6	70	13	13.2	
7	70	13	9.7	
8	80	15	11.3	
9	80	15	15.6	
10				
11				

Data Science: Python Pandas DataFrame from Excel

STEPS:

1. import the pandas library
2. Create a variable for the data frame to store all columns and values from the Excel worksheet
3. Use the API from pandas
`pandas.read_excel("filename.xlsx")`
to read the file "filename.xlsx" and assign all columns and values to the data frame variable

Data Science: Python Pandas DataFrame from Excel

```
In [19]: import pandas as pd  
  
df = pd.read_excel ("QtyDemand.xlsx")  
  
print(df)
```

	Price	Sales	Quantity
0	50	9	7.1
1	50	9	5.7
2	60	11	5.7
3	60	11	6.6
4	70	13	13.2
5	70	13	9.7
6	80	15	11.3
7	80	15	15.6

```
In [ ]:
```

Data Science: Python Pandas DataFrame from Excel

- You can access the specific column by referencing the index (column label) of that column
 - **df["Price"]** will return the values for the entire column "Price"
 - You can also use the form **df.Price**
- 0, 1, 2, 3, 4, 5, 6, 7 on the left most column is the index for the rows. You can access the value stored in column Price row 0 using **df.Price[index]**
 - **df.Price[0]** will return the value of the first element of column "Price" = 50
 - **df.Price[6]** will return the value of the seventh element of column "Price" = 80
 - **df.Sales[2]** = 11
 - **df.Quantity[4]** = 13.2

```
In [19]: import pandas as pd

df = pd.read_excel ("QtyDemand.xlsx")

print(df)
```

	Price	Sales	Quantity
0	50	9	7.1
1	50	9	5.7
2	60	11	5.7
3	60	11	6.6
4	70	13	13.2
5	70	13	9.7
6	80	15	11.3
7	80	15	15.6

```
In [ ]:
```

Data Science: Python Pandas DataFrame from Excel

```
In [22]: print(df.Price[0])
```

50

```
In [23]: print(df.Price[6])
```

80

```
In [24]: print(df.Sales[2])
```

11

```
In [25]: print(df.Quantity[4])
```

13.2

```
In [21]: print(df["Price"])
```

0 50

1 50

2 60

3 60

4 70

5 70

6 80

7 80

Name: Price, dtype: int64

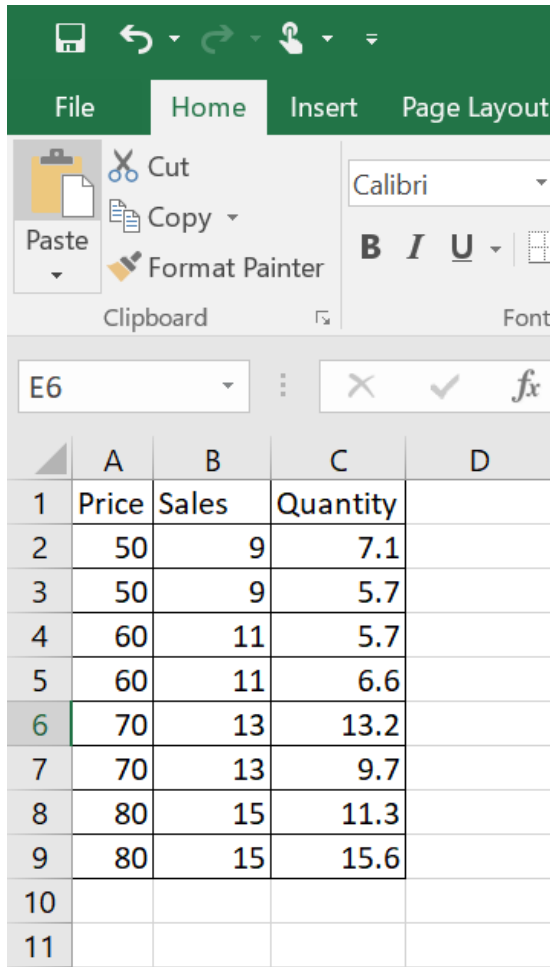
Data Science: Python Pandas DataFrame from Excel Columns



	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

```
pandas.read_excel("filename.xlsx")
```

Data Science: Python Pandas DataFrame from Excel



The screenshot shows the Microsoft Excel interface. The 'Home' tab is selected in the ribbon. The 'Clipboard' group shows 'Cut', 'Copy', 'Paste', and 'Format Painter' options. The 'Font' group shows 'Calibri' font, bold (B), italic (I), and underline (U) options. The active cell is E6. The spreadsheet data is as follows:

	A	B	C	D
1	Price	Sales	Quantity	
2	50	9	7.1	
3	50	9	5.7	
4	60	11	5.7	
5	60	11	6.6	
6	70	13	13.2	
7	70	13	9.7	
8	80	15	11.3	
9	80	15	15.6	
10				
11				

What if we only want to read column
“Price” into the dataframe variable?

Data Science: Python Pandas DataFrame from Excel

STEPS:

1. import the pandas library
2. Create a variable for the data frame to store all columns and values from the Excel worksheet
3. Use the API from pandas
`pandas.read_excel("filename.xlsx")` to read the file "filename.xlsx" and assign all columns and values to the data frame variable.
 1. This time we will use an additional argument named `usecols`.
 2. `pandas.read_excel("filename.xlsx", usecols=[0])` to read first column only
 3. `pandas.read_excel("filename.xlsx", usecols=[0, 1])` to read first column and second column only

Data Science: Python Pandas DataFrame from Excel

```
import pandas as pd

df = pd.read_excel ("QtyDemand.xlsx", usecols = [0])

print(df)
```

	Price
0	50
1	50
2	60
3	60
4	70
5	70
6	80
7	80

```
import pandas as pd

df = pd.read_excel ("QtyDemand.xlsx", usecols = [0, 2])

print(df)
```

	Price	Quantity
0	50	7.1
1	50	5.7
2	60	5.7
3	60	6.6
4	70	13.2
5	70	9.7
6	80	11.3
7	80	15.6

Data Science: Python Pandas DataFrame Saving to Excel File

```
import pandas as pd  
df = pd.read_excel ("QtyDemand.xlsx", usecols = [0])  
print(df)
```

	Price
0	50
1	50
2	60
3	60
4	70
5	70
6	80
7	80

df.to_excel ("test2.xlsx")

An Excel spreadsheet with columns A, B, and C. Column A contains indices 0-7, column B contains the word 'Price', and column C contains values 50-80. A red circle highlights the range A1:B8.

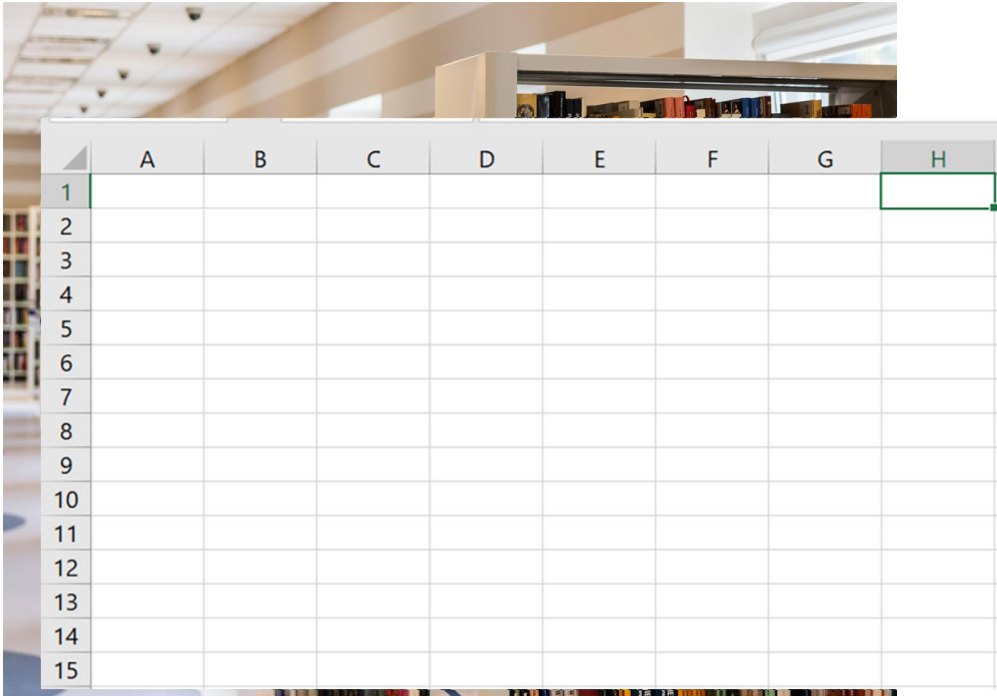
	A	B	C
1		Price	
2	0		50
3	1		50
4	2		60
5	3		60
6	4		70
7	5		70
8	6		80
9	7		80

df.to_excel ("test2.xlsx", index = False)

An Excel spreadsheet with columns A and B. Column A contains the word 'Price' and column B contains values 50-80. The index is not present.

A	B
Price	
	50
	50
	60
	60
	70
	70
	80
	80

Data Science: Python Pandas DataFrame from csv



	A	B	C	D	E	F	G	H
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								

Data Science: Python Pandas DataFrame from csv

```
Price,Sales,Quantity
```

```
50,9,7.1
```

```
50,9,5.7
```

```
60,11,5.7
```

```
60,11,6.6
```

```
70,13,13.2
```

```
70,13,9.7
```

```
80,15,11.3
```

```
80,15,15.6
```

Data Science: Python Pandas DataFrame from csv

STEPS:

1. import the pandas library
2. Create a variable for the data frame to store all columns and values from the csv file
3. Use the API from pandas `pandas.read_csv("filename.csv")` to read the file "filename.csv" and assign all data to the data frame variable

Data Science: Python Pandas DataFrame from csv

```
import pandas as pd

df = pd.read_csv ("QtyDemand.csv")

print(df)
```

	Price	Sales	Quantity
0	50	9	7.1
1	50	9	5.7
2	60	11	5.7
3	60	11	6.6
4	70	13	13.2
5	70	13	9.7
6	80	15	11.3
7	80	15	15.6

Data Science: Python Pandas DataFrame from csv

```
import pandas as pd

df = pd.read_csv ("QtyDemand.csv", usecols = [0])

print(df)
```

Specific Column

	Price
0	50
1	50
2	60
3	60
4	70
5	70
6	80
7	80

Data Science: Python Pandas DataFrame Saving to a csv file

```
import pandas as pd  
df = pd.read_csv ("QtyDemand.csv", usecols = [0])  
print(df)
```

	Price
0	50
1	50
2	60
3	60
4	70
5	70
6	80
7	80

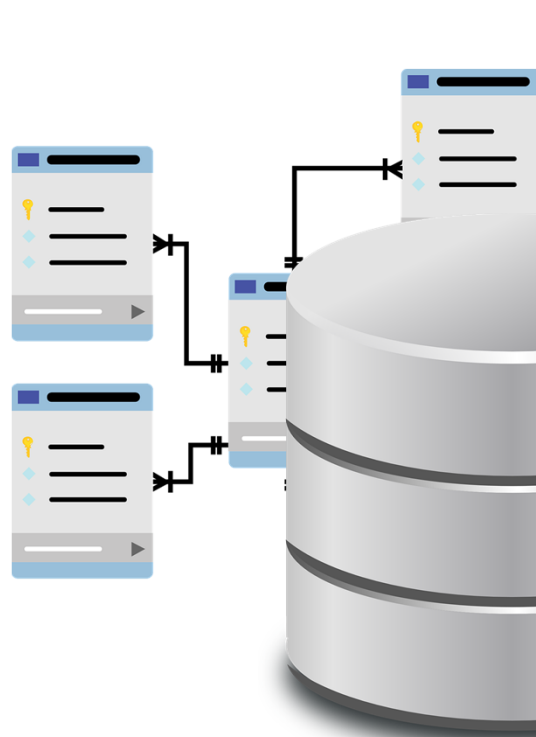
df.to_csv ("test3.csv")

df.to_csv ("test3.csv", index = False)

A	B
Price	
50	
50	
60	
60	
70	
70	
80	
80	

	A	B	C
1		Price	
2	0	50	
3	1	50	
4	2	60	
5	3	60	
6	4	70	
7	5	70	
8	6	80	
9	7	80	

Introduction to Data Science: CSV Import Using MySQL Workbench



The diagram illustrates the MySQL Workbench interface. On the left, there are three panels representing the 'Servers' tree, 'Query History', and 'Query Log'. In the center, a large 3D cylinder represents the database. To the right, a table is displayed with 7 columns and 22 rows of data. The table is titled 'Sales' and contains data for dates from 1/1/2019 to 1/21/2019.

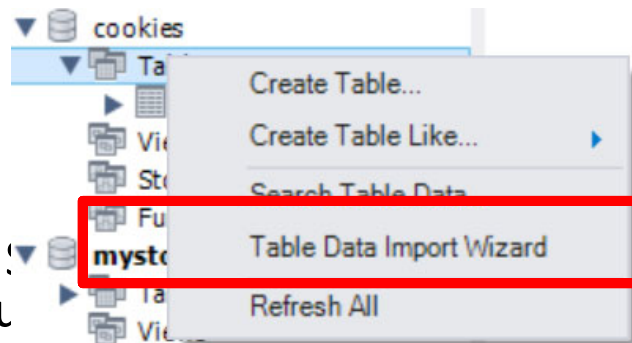
	A	B	C	D	E	F	G
1	Sales_Date	Day_of_W	Salesman	Temperatu	Tweets	Price	Sales
2	1/1/2019	Tuesday	John	72	2	0.5	177
3	1/2/2019	Wednesd	John	82	3	0.5	127
4	1/3/2019	Thursday	John	69	5	0.5	172
5	1/4/2019	Friday	John	100	7	0.5	150
6	1/5/2019	Saturday	Ada	69	6	0.3	103
7	1/6/2019	Sunday	Ada	91	8	0.5	120
8	1/7/2019	Monday	Ada	81	3	0.3	96
9	1/8/2019	Tuesday	John	88	6	0.5	150
10	1/9/2019	Wednesd	John	69	8	0.5	177
11	1/10/2019	Thursday	John	61	10	0.5	190
12	1/11/2019	Friday	John	79	1	0.5	154
13	1/12/2019	Saturday	John	94	9	0.5	160
14	1/13/2019	Sunday	John	80	5	0.5	161
15	1/14/2019	Monday	John	64	8	0.5	185
16	1/15/2019	Tuesday	Ada	94	6	0.5	137
17	1/16/2019	Wednesd	Ada	68	6	0.3	106
18	1/17/2019	Thursday	Ada	74	2	0.3	85
19	1/18/2019	Friday	John	89	4	0.5	122
20	1/19/2019	Saturday	John	87	6	0.5	187
21	1/20/2019	Sunday	John	100	7	0.5	165
22	1/21/2019	Monday	John	89	8	0.5	158

Introduction to Data Science: CSV Import Using MySQL Workbench

- Step 1
 - CREATE SCHEMA cookies;
- Step 2

- Step 3

- Convert !
date valu



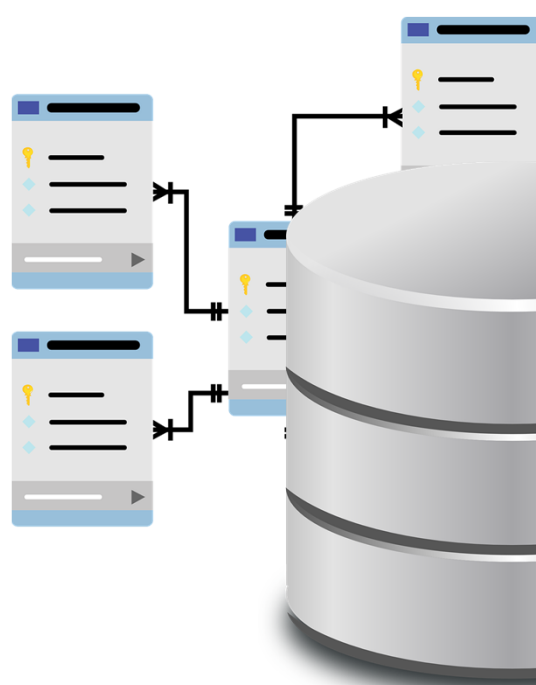
nt to do so or your data include a

SET SQL_SAFE_UPDATES=0;

UPDATE cookies.`cookies sample`

SET Sales_Date = **str_to_date**(Sales_Date, '%m/%d/%Y');

SQL Load CSV Data and Covert Date



The diagram illustrates a data loading process. On the left, there are three server-like icons representing data sources. Arrows point from these sources to a central database cylinder. One arrow points directly to the cylinder, while the other two pass through a small rectangular box with a lightbulb icon, suggesting a transformation or processing step before the data is loaded into the database.

	A	B	C	D	E	F	G
1	Sales_Date	Day_of_W	Salesman	Temperatu	Tweets	Price	Sales
2	1/1/2019	Tuesday	John	72	2	0.5	177
3	1/2/2019	Wednesd	John	82	3	0.5	127
4	1/3/2019	Thursday	John	69	5	0.5	172
5	1/4/2019	Friday	John	100	7	0.5	150
6	1/5/2019	Saturday	Ada	69	6	0.3	103
7	1/6/2019	Sunday	Ada	91	8	0.5	120
8	1/7/2019	Monday	Ada	81	3	0.3	96
9	1/8/2019	Tuesday	John	88	6	0.5	150
10	1/9/2019	Wednesd	John	69	8	0.5	177
11	1/10/2019	Thursday	John	61	10	0.5	190
12	1/11/2019	Friday	John	79	1	0.5	154
13	1/12/2019	Saturday	John	94	9	0.5	160
14	1/13/2019	Sunday	John	80	5	0.5	161
15	1/14/2019	Monday	John	64	8	0.5	185
16	1/15/2019	Tuesday	Ada	94	6	0.5	137
17	1/16/2019	Wednesd	Ada	68	6	0.3	106
18	1/17/2019	Thursday	Ada	74	2	0.3	85
19	1/18/2019	Friday	John	89	4	0.5	122
20	1/19/2019	Saturday	John	87	6	0.5	187
21	1/20/2019	Sunday	John	100	7	0.5	165
22	1/21/2019	Monday	John	89	8	0.5	158

SQL Load Data from CSV File

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
▶	1/1/2019	Tuesday	John	72	2	0.5	177
	1/2/2019	Wednesday	John	82	3	0.5	127
	1/3/2019	Thursday	John	69	5	0.5	172
	1/4/2019	Friday	John	100	7	0.5	150
	1/5/2019	Saturday	Ada	69	6	0.3	103
	1/6/2019	Sunday	Ada	91	8	0.5	120
	1/7/2019	Monday	Ada	81	3	0.3	96
	1/8/2019	Tuesday	John	88	6	0.5	150

sales 1 x

CREATE TABLE cookies.sales

(Sales_Date **varchar(10)**,

Day_of_Week **varchar(10)**,

Salesman **varchar(10)**,

Temperature **INT**,

Tweets **INT**,

Price **FLOAT**,

Sales **INT**);

SQL Load Data from CSV File

```
CREATE SCHEMA cookies;
```

```
CREATE TABLE cookies.sales  
(Sales_Date varchar(10),  
Day_of_Week varchar(10),  
Salesman varchar(10),  
Temperature INT,  
Tweets INT,  
Price FLOAT,  
Sales INT);
```

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Cookies Sample.csv'  
INTO TABLE cookies.sales  
FIELDS TERMINATED BY ','  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

```
(@Sales_Date, Day_of_Week, Salesman, Temperature, Tweets, Price, Sales)  
SET Sales_Date = STR_TO_DATE(@Sales_Date, '%m/%d/%Y');
```

SQL Load Data from CSV File

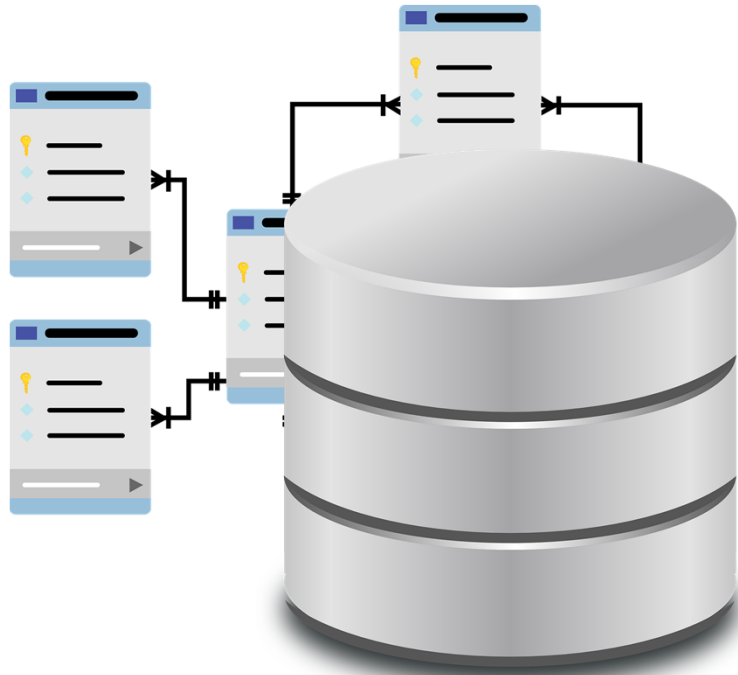
	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
▶	1/1/2019	Tuesday	John	72	2	0.5	177
	1/2/2019	Wednesday	John	82	3	0.5	127
	1/3/2019	Thursday	John	69	5	0.5	172
	1/4/2019	Friday	John	100	7	0.5	150
	1/5/2019	Saturday	Ada	69	6	0.3	103
	1/6/2019	Sunday	Ada	91	8	0.5	120
	1/7/2019	Monday	Ada	81	3	0.3	96
	1/8/2019	Tuesday	John	88	6	0.5	150
sales 1 ×							

Before

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
▶	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

After

Introduction to Data Science: Data Manipulation Language



Introduction to Data Science: Data Manipulation Language

`SELECT column from table`

- select data from table

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

Introduction to Data Science: Data Manipulation Language

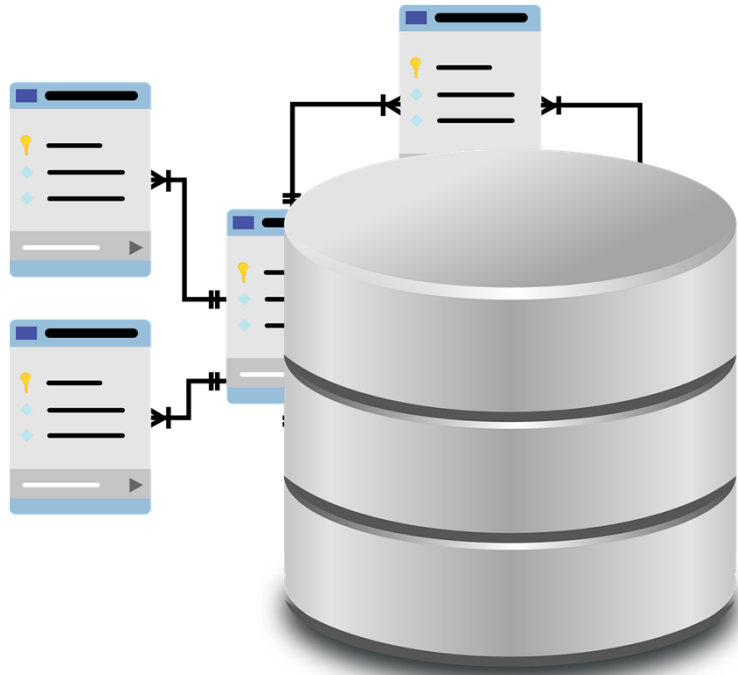
SELECT *column* FROM *table* WHERE *expression*

Expression in the form of *column* operator *value*

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

- select data from table based on the condition specified
- Operator =, <, >, <=, >=

Introduction to Data Science: DML SELECT AND OR IN LIKE



Introduction to Data Science: Data Manipulation Language

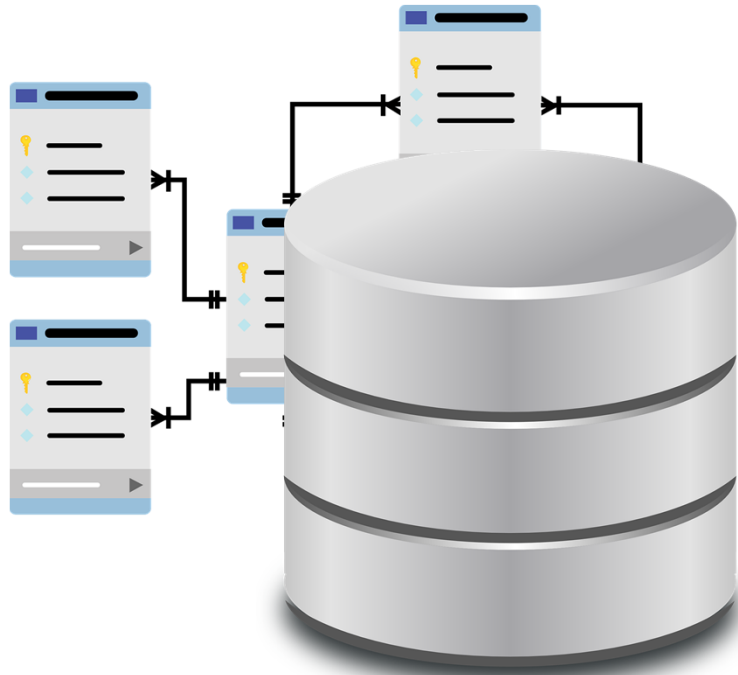
SELECT *column* FROM *table* WHERE *expression*

Expression in the form of *column* operator *value*

- select data from table based on the condition specified
- Operator AND, OR, IN, NOT IN, LIKE, NOT LIKE

last_name	first_name	county	district	school	primary_job	fte	salary	certificate
Heckman	William	Atlantic	Atlantic City	Pennsylvania Ave School	Mathematics Grades 5 - 8	1	98774	Standard o
Bird	Kelly	Atlantic	Atlantic City	Atlantic City High School	Coordinator Substance Abuse	1	118415	Standard o
Bean	David B	Atlantic	Atlantic City	Atlantic City High School	Health & Physical Education	0.8	98774	Standard o
Campo	Paula Mia	Atlantic	Atlantic City	Atlantic City High School	Resource Program In-class	1	66184	Standard o
Adams-meyer	Della L	Atlantic	Atlantic City	Atlantic City High School	School Psychologist	1	101866	Standard o
Mansor	Theresa	Atlantic	Atlantic City	Atlantic City High School	Resource Program In-class	1	98774	Standard o
Mendez	Cheryl	Atlantic	Atlantic City	Atlantic City High School	Health & Physical Education	1	109584	Standard o
Toland	John	Atlantic	Atlantic City	Atlantic City High School	Health & Physical Education	1	51696	Standard o

Introduction to Data Science: Data Manipulation Language



Introduction to Data Science: Data Manipulation Language

SELECT *column* FROM *table* ORDER BY *column* DESC

- select data from table and sort the result in ascending or descending order

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

Introduction to Data Science: Data Manipulation Language

SELECT *column* FROM *table* WHERE *column*
BETWEEN *value* AND *value*

- Select data from table based on the interval specified

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
▶	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

Introduction to Data Science: Data Manipulation Language

SET SQL_SAFE_UPDATES = 0;

UPDATE *table*

SET *column*=value [, *column*=value,...]

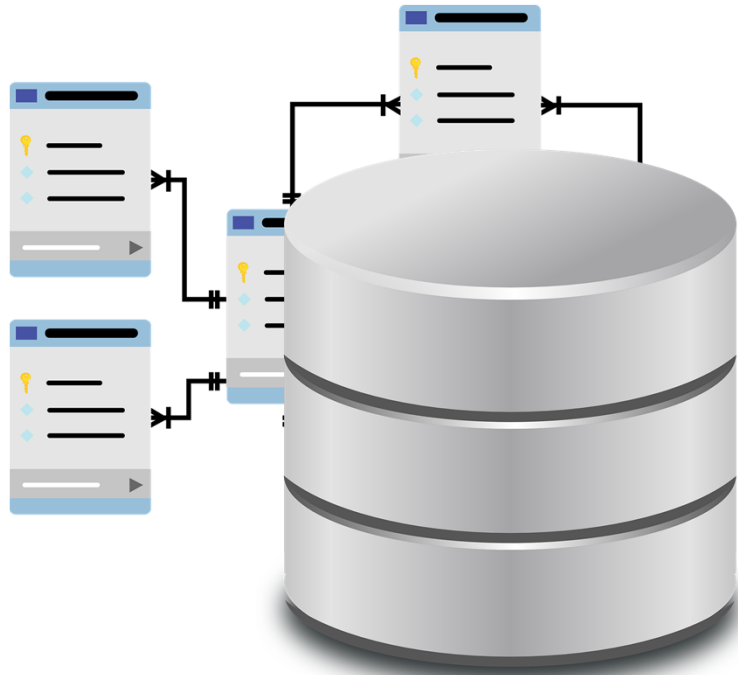
WHERE *expression*

Expression in the form of *column* = *value*

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

- Update table based on the condition specified

Introduction to Data Science: Data Manipulation Language



Introduction to Data Science: Data Manipulation Language

SELECT MAX(*column*) FROM *table*

SELECT AVG(*column*) FROM *table*

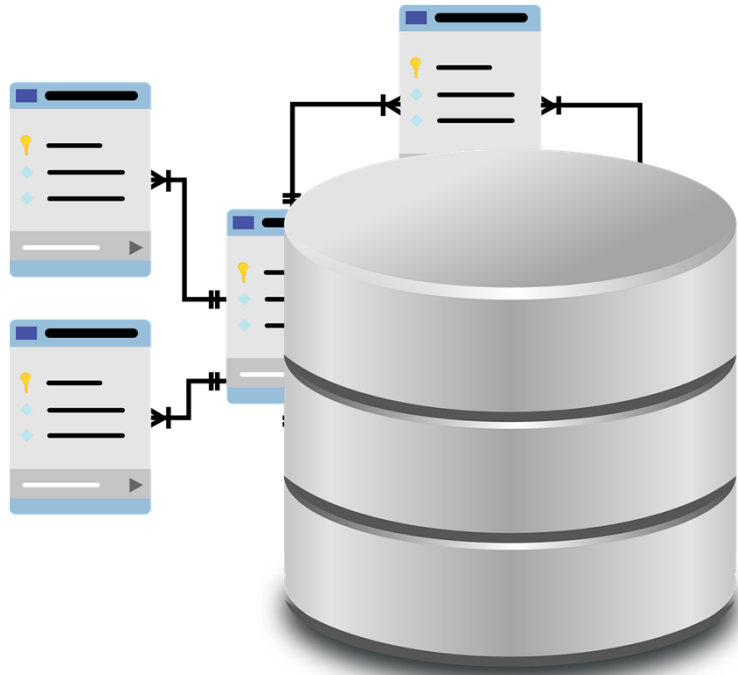
SELECT MIN(*column*) FROM *table*

SELECT COUNT(*column*) FROM *table*

SELECT SUM(*column*) FROM *table*

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

Introduction to Data Science: Data Manipulation Language



Introduction to Data Science: Data Manipulation Language

SELECT *column* FROM *table* ORDER BY *column* DESC

- select data from table and sort the result in ascending or descending order

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

Introduction to Data Science: Data Manipulation Language

SELECT *column* FROM *table* WHERE *column*
BETWEEN *value* AND *value*

- Select data from table based on the interval specified

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

Introduction to Data Science: Data Manipulation Language

SET SQL_SAFE_UPDATES = 0;

UPDATE *table*

SET *column*=value [, *column*=value,...]

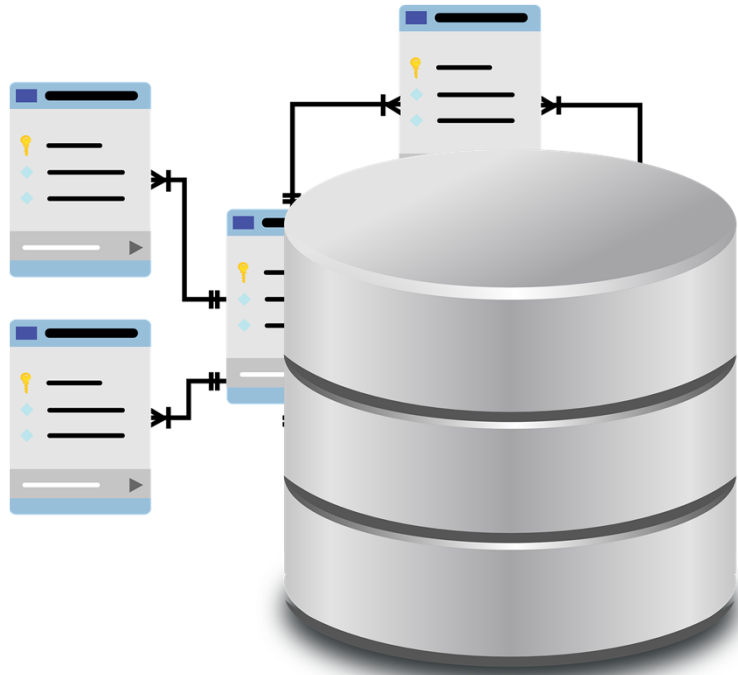
WHERE *expression*

Expression in the form of *column* = *value*

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150

- Update table based on the condition specified

Introduction to Data Science: Data Manipulation Language



Introduction to Data Science: Data Manipulation Language

SELECT MAX(*column*) FROM *table*

SELECT AVG(*column*) FROM *table*

SELECT MIN(*column*) FROM *table*

SELECT COUNT(*column*) FROM *table*

SELECT SUM(*column*) FROM *table*

	Sales_Date	Day_of_Week	Salesman	Temperature	Tweets	Price	Sales
►	2019-01-01	Tuesday	John	72	2	0.5	177
	2019-01-02	Wednesday	John	82	3	0.5	127
	2019-01-03	Thursday	John	69	5	0.5	172
	2019-01-04	Friday	John	100	7	0.5	150
	2019-01-05	Saturday	Ada	69	6	0.3	103
	2019-01-06	Sunday	Ada	91	8	0.5	120
	2019-01-07	Monday	Ada	81	3	0.3	96
	2019-01-08	Tuesday	John	88	6	0.5	150